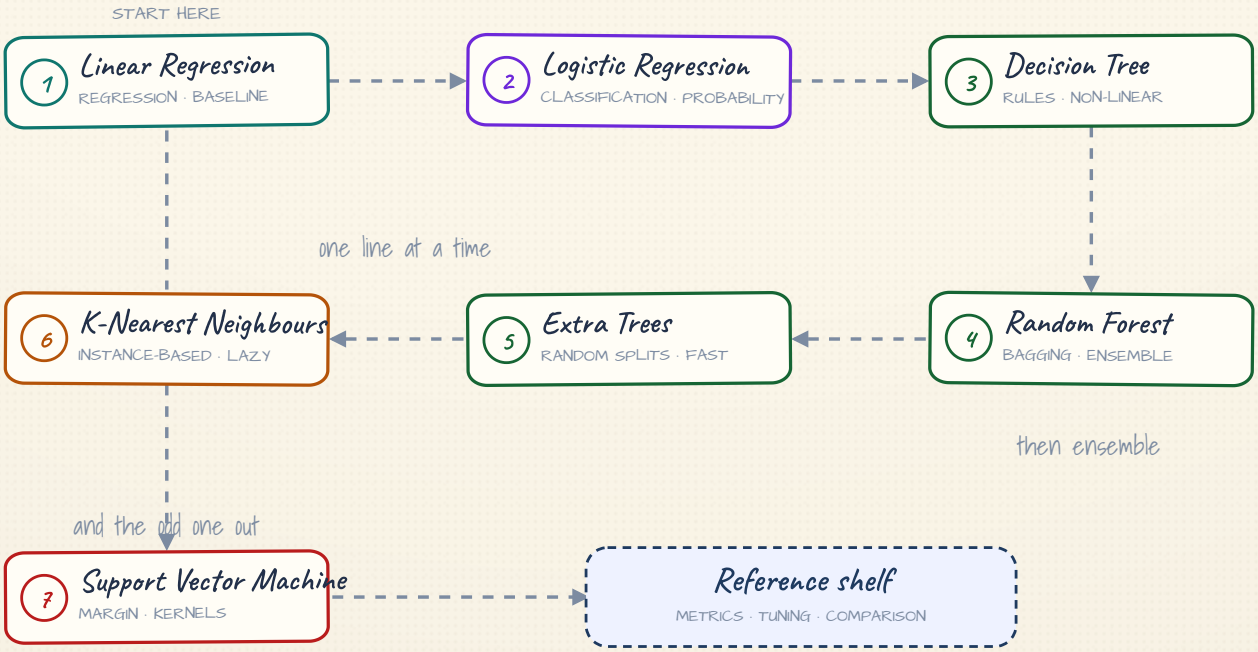
ML 01

Machine Learning Models

A handwritten field guide — from the first best-fit line to the last support vector.



NOTES BY
Chandan Chaudhari
github.com/chandanc5525

7 models · 49 pages
Linear → Logistic → Trees → KNN → SVM
Written November 2026

These are the notes I wish I had when I started: one model per chapter, always in the same order, so that comparing them later is effortless. Every chapter answers the same nine questions.

1 THE NINE-PART CHAPTER TEMPLATE

same order, every chapter

- **Overview** — the model in one paragraph, plus its type.
- **Introduction & purpose** — the problem it was invented for.
- **Intuition** — the idea in plain language, before any maths.
- **How it works** — concepts, formulas, training recipe.
- **Assumptions** — what must be true for it to work well.
- **Evaluation metrics** — what to measure, and when to use each.
- **Use cases** — where it earns its keep in the real world.
- **Advantages, limits, mistakes** — including the traps.
- **Summary** — a compact card and comparison table.

READ ME FIRST

Short on time? Start with the **visual map** on page 5, the **one-page picker** on page 45, and the **comparison table** on page 44.

Then return to a chapter when you need the maths or the metrics.

7
models, each with the same nine sections

2
metric cheat sheets: regression & classification

2 THE VISUAL LANGUAGE OF THESE NOTES

DEFINITION

A concept worth memorising. Yellow, taped, slightly crooked.

TIP

Practical advice that saves time or prevents a quiet bug.

WARNING

The mistakes that look harmless and cost accuracy.

FORMULA

$$RMSE = \sqrt{\left(\Sigma(y_i - \hat{y}_i)^2 / n \right)}$$

REMEMBER

Every boxed item is either a definition, a formula, or a warning. Boxed maths is always followed by a plain-English gloss.

3 NOTATION USED THROUGHOUT

the alphabet of the guide

SYMBOL	MEANING	SYMBOL	MEANING
n	*number of rows (samples)*	e_i	*residual* = $y_i - \hat{y}_i$
p	*number of features (columns)*	α / λ	*regularisation strength (larger = simpler)*
X	*feature matrix,* $n \times p$	C	*inverse regularisation in SVM / logistic*
y, $\hat{y}$	*true target, model prediction*	k	*number of neighbours or CV folds*
β, w, b	*weights / coefficients, intercept*	σ^2	*variance of the irreducible noise*

Subscript i always indexes a row; subscript j always indexes a feature. A hat ($\hat{y}$) means "the model's estimate".

A study guide is only trustworthy if you know exactly what it covers. So here is the audit trail — what the folder contained, what I assumed, and what is deliberately out of scope.

3 WHAT THE FOLDER ACTUALLY CONTAINED

verified on disk

FOLDER	`MachineLearninfg_Models` — note the extra "g" in the repository spelling.
CONTENTS	Seven empty sub-folders and one empty `__init__.py`. No notebooks, datasets, notes or code were present.
MODEL FOLDERS	LinearRegression · LogisticRegression · DecisionTree · RandomForest · ExtraTree · KNN · SVM
CONSEQUENCE	The scope below is inferred from those seven names. No eighth model was invented.

4 EXPLICIT ASSUMPTIONS

stated, not hidden

- "**All Linear Regression Models**" is read as OLS linear regression plus its regularised variants **Ridge, Lasso and Elastic Net**. They share the same linear predictor; only the penalty changes. They are clearly flagged as extensions, not as extra folders.
- "**ExtraTree**" means **Extremely Randomised Trees**, the ensemble — not a single extra-split tree.
- **KNN** and **SVM** are taught as classifiers first, because that is their common use; their regression forms (KNN regression, SVR) are described inside the same chapter.
- Both tree ensembles (**Random Forest**, **Extra Trees**) are presented for classification and regression.
- **Clustering** is explained on page 5 and its metrics on page 43 purely as a contrast, because the folder has no clustering model. No clustering algorithm is taught as if it were in scope.
- Hyper-parameters and defaults refer to **scikit-learn** conventions, the library this repository already depends on.
- All numerical examples are small and hand-checkable; they are illustrative, not fitted to a dataset in the repository.

5 HOW TO STUDY A CHAPTER

active recall

1. Read the overview and the intuition. Close the notes and explain the model out loud in two sentences.
2. Re-derive the boxed formula on paper. If you cannot, the mechanics are not yet yours.
3. Check the assumptions against a dataset you know. Which one would break first?
4. Pick the metric *before* looking at results, then ask what a false positive costs.
5. Recite one advantage, one limitation, and one common mistake without looking.

THE TWO-SENTENCE TEST

If you cannot say *what the model optimises* and *what shape its decision surface has*, you know the API but not the model. Those two facts explain almost every strength and weakness in this guide.

AUTHOR

Chandan Chaudhari

GitHub: `github.com/chandanc5525`

Notes compiled for revision and teaching. Corrections and improvements are welcome via GitHub.

PART 0 · GETTING ORIENTED

2 How to use these notes *template & visual language* 2

3 Scope, sources & assumptions *what the folder contained* 3

4 Contents *the shape of the guide* 4

5 The machine-learning landscape *regression vs classification vs clustering* 5

6 Workflow, data splits & cross-validation *how every model is judged* 6

7 Bias, variance & regularisation *the engine behind every fix* 7

PART 1 · LINEAR MODELS

8 Linear Regression *OLS · diagnostics · Ridge / Lasso / Elastic Net* 8

14 Logistic Regression *sigmoid · log-loss · thresholds* 14

PART 2 · TREE-BASED MODELS

18 Decision Tree *recursive splitting · Gini · pruning* 18

22 Random Forest *bagging · OOB · feature importance* 22

26 Extra Trees *random thresholds · speed vs bias* 26

PART 3 · DISTANCE & MARGIN MODELS

30 K-Nearest Neighbours *similarity · k · curse of dimensionality* 30

34 Support Vector Machines *max margin · C · kernels* 34

PART 4 · THE REFERENCE SHELF

39 Regression metrics *MAE · RMSE · R^2 · residual checks* 39

40 Classification metrics *precision · recall · F1 · cost* 40

41 ROC, PR & thresholds *curves · operating points* 41

42 Calibration & cost *reliability · Platt · isotonic* 42

43 Clustering metrics *silhouette · elbow · ARI (contrast only)* 43

44 Master comparison table *all seven models, side by side* 44

45 Which model should I pick? *a one-page decision map* 45

46 Hyper-parameters & tuning *what to turn, and in what order* 46

47 Common mistakes & audit *twelve traps, four checklists* 47

48 Glossary *the vocabulary, in plain English* 48

49 One-page revision sheet *everything, condensed* 49

Page numbers refer to the printed folio in each footer. Chapters are self-contained: each one repeats the nine-part structure, so you can jump straight to the model you need.

Before the models, the map. Almost every confusion in machine learning starts with mixing up what is being predicted — a number, a class, or a grouping.

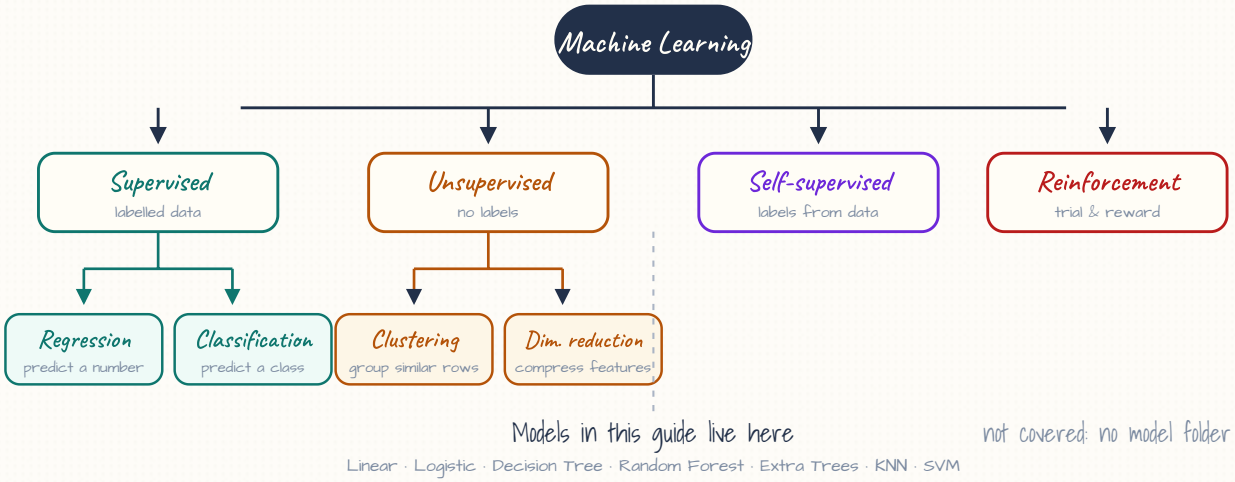


Fig. 4.1 — The four learning settings, and where the seven models of this guide sit. Only the supervised branch is covered in depth.

1 THE THREE QUESTIONS THAT SEPARATE MODEL TYPES

the distinction that matters

QUESTION	REGRESSION	CLASSIFICATION	CLUSTERING
Target	*A continuous number: price, demand, temperature*	*A discrete label: fraud / not fraud, species*	*No target at all — labels are discovered*
Output	*$\hat{y}$ (a quantity) and its error*	*A class and/or probability $p(y=1\|x)$*	*A group id per row, plus centroids or density*
Supervision	*Labelled data*	*Labelled data*	*Unsupervised*
Typical metrics	*MAE, RMSE, R^2*	*Precision, recall, F1, ROC-AUC*	*Silhouette, Davies–Bouldin, ARI*
In this guide	*Linear Regression, KNN reg., SVR, tree regressors*	*Logistic Regression, KNN, SVM, tree classifiers*	*Concept only — no model folder*

2 DEFINITIONS WORTH PINNING DOWN

SUPERVISED LEARNING

The dataset contains the answer. The model learns a mapping $f: X \to y$ from examples, then predicts for new rows.

REGRESSION

Supervised learning where y is numeric and *ordered*. Predicting 3.2 when the truth is 3.0 is a small miss; that notion of distance is central.

CLASSIFICATION

Supervised learning where y is a category. "Cat" is not between "dog" and "horse"; there is no distance, so accuracy and confusion matrices replace error.

CLUSTERING

Unsupervised grouping of similar rows. There is no correct answer to compare against, which is why its evaluation (page 43) is a different sport entirely.

3 WHERE THE SEVEN MODELS BELONG

all seven, located

- **Linear Regression** — regression; interpretable baseline.
- **Logistic Regression** — classification; linear log-odds.
- **Decision Tree / Forest / Extra Trees** — both; rule-based ensembles.
- **KNN** — both; predictions from the nearest rows.
- **SVM** — both; widest margin, kernels for curves.
- **Ridge / Lasso / Elastic Net** — penalised linear regression.

Every model in this guide is judged the same way: fit on training data, choose settings on validation data, and report once on data the model has never seen. Break that discipline and the score becomes fiction.

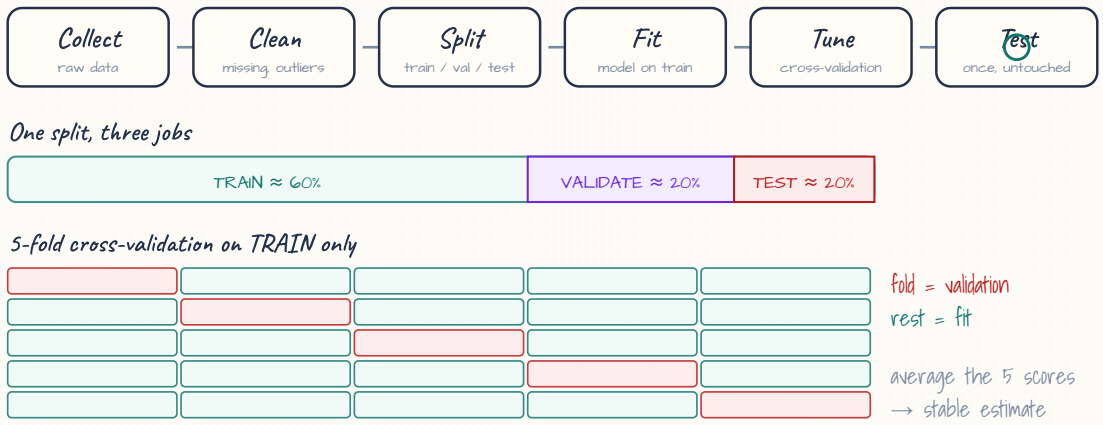


Fig. 5.1 — The workflow used throughout these notes. Cross-validation happens inside the training block; the test row is opened once, at the very end.

1 THREE JOBS, THREE DATASETS

TRAIN
The model sees these rows and adjusts its parameters. 60–80% of the data.

VALIDATION
Used to compare models and tune hyper-parameters. Never used to fit coefficients.

TEST
Touched once, after all decisions. Its score is the honest estimate you report.

2 CROSS-VALIDATION: A MORE STABLE VERDICT

k = 5 or 10

Split the training set into k folds (5 or 10 is standard), train on $k-1$ and validate on the held-out fold, then rotate. The reported score is the **mean ± standard deviation** across folds — the spread matters as much as the mean.

- **Stratified k-fold** for classification: keep the class ratio in every fold, or a rare class may vanish from one of them.
- **TimeSeriesSplit** for anything ordered in time: always train on the past and validate on the future, never shuffle.
- **GroupKFold** when rows are not independent (several visits per patient): keep a group entirely in one fold.

3 DATA LEAKAGE: THE SILENT INFLATION OF SCORES

check before trusting any number

LEAKS THAT FLATTER YOU

- Scaling, imputing or selecting features *before* the split — use a Pipeline.
- Target encoding computed on the full dataset.
- Duplicate or near-duplicate rows split across train and test.
- Using future information (next month's balance) to predict this month.
- Tuning on the test set — even "just once more".

A SANITY LADDER

1. Beat a dumb baseline (mean for regression, majority class for classification).
2. Beat it by a margin larger than the fold-to-fold spread.
3. Check the learning curve: more data should help, not hurt.
4. Inspect errors by segment, not just the aggregate score.

Why does a model that is perfect on training data fail tomorrow? The answer is a budget: every prediction error is paid for in bias, variance, or irreducible noise.

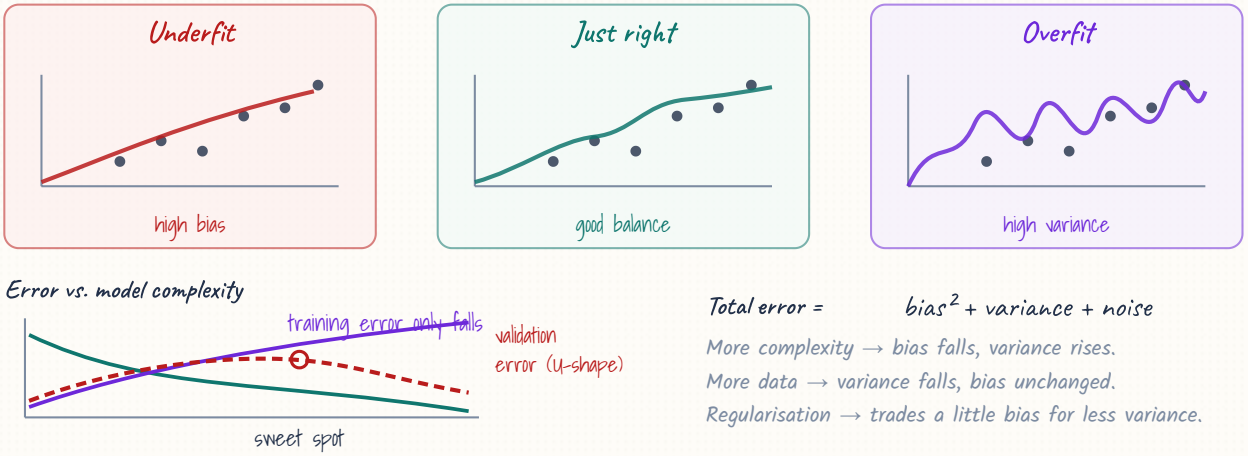


Fig. 6.1 – Underfitting and overfitting are two ends of one dial. Validation error, not training error, tells you which side of the dial you are on.

1 THE DECOMPOSITION, IN WORDS

EXPECTED SQUARED ERROR

$$E[(y - \hat{y})^2] = \text{bias}^2 + \text{variance} + \sigma^2_{\text{noise}}$$

Bias = error from a model that is too simple to represent the truth. ***Variance*** = error from sensitivity to the particular training sample. ***Noise*** = the part no model can remove.

High bias (underfit): training *and* validation error are both high and close together. The model has not learned the pattern.

High variance (overfit): training error is tiny, validation error is much larger. The model memorised the sample, noise included.

2 THE STANDARD REMEDIES

fix the cause, not the score

SYMPTOM	WHAT YOU SEE	WHAT TO TRY, IN ORDER
Underfit	*High train & validation error*	*Richer features, interactions, non-linear model, less regularisation, longer training*
Overfit	*Train error ↓↓, validation error ↑*	*More data, stronger regularisation, simpler model, early stopping, pruning, feature selection*
Both	*Validation noisy across folds*	*More folds, repeated CV, better metric, check for leakage*

3 REGULARISATION: PAYING BIAS TO BUY CALM

the most useful dial

Add a penalty for large coefficients to the loss. One knob, α (or $C = 1/\alpha$ in SVM and logistic regression), controls the trade: larger penalty → simpler model → more bias, less variance.

THE THREE PENALTIES

$$\text{L2: } J + \alpha\Sigma\beta_j^2 \quad \cdot \quad \text{L1: } J + \alpha\Sigma|\beta_j| \quad \cdot \quad \text{Elastic: } J + \alpha(\rho\Sigma|\beta_j| + (1-\rho)\Sigma\beta_j^2)$$

L2 shrinks smoothly · L1 can set coefficients to exactly zero (feature selection) · Elastic Net mixes both. Geometry on page 11.

TUNE IT, DON'T GUESS IT

Tune α in the validation loop on a log grid, using the same folds for every model.

PART I · LINEAR MODELS · CHAPTER 01

Linear Regression

The oldest supervised model, and still the one to beat: fit the straight line that leaves the smallest total squared miss.

SUPERVISED | REGRESSION | PARAMETRIC | LINEAR IN COEFFICIENTS | PAGES 7-12

1 OVERVIEW — what it is

Linear regression predicts a **continuous number** — a price, a demand, a temperature — as a weighted sum of input features. One feature gives a line of best fit; several give a flat plane (a hyperplane). "Learning" here means estimating those weights.

> THE MODEL, IN ONE LINE
>
> $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p + \varepsilon$ — each β_j is how much y moves when x_j moves one unit and everything else stays fixed.

2 INTRODUCTION & PURPOSE — why it exists

Francis Galton noticed in 1886 that tall parents tend to have children a little closer to average height — he called it *regression to the mean*, and the name stuck to the method. The purpose has not changed: describe how an outcome moves with its inputs, with as few assumptions as possible.

- **Baseline first.** Any fancy model must beat this before it earns its complexity.
- **Quantify effects.** A coefficient plus confidence interval is an answer, not just a prediction.
- **Forecast.** Extrapolate a trend when the relationship is genuinely stable.
- **Screen & adjust.** Rank features, or remove the effect of a confounder before comparison.

3 INTUITION — least squares, visually

Imagine a rigid ruler and a scatter of nails on a board. The ruler cannot pass through every nail, so you slide and tilt it until the sum of squared vertical gaps is as small as possible. That is the whole idea.

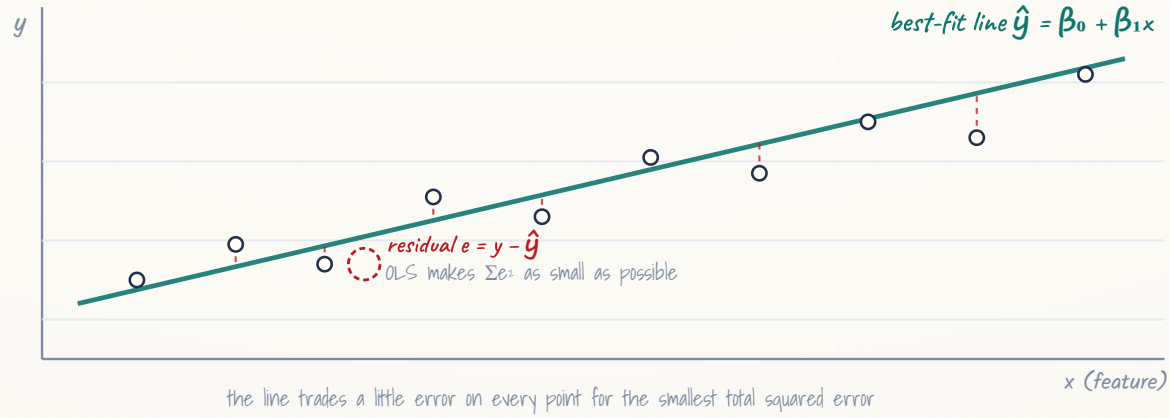


Fig. 7.1 — The line is chosen by the data, not drawn by hand. Each dashed gap is a residual; squaring makes one large miss cost more than several small ones.

Squaring is not arbitrary: it makes the solution unique, differentiable, and equivalent to assuming the errors are roughly bell-shaped. It also means **outliers pull hard** on the line — a single badly measured point can tilt the whole fit.

4 How it works — the engine

With n rows and p features, stack the inputs into a matrix X (with a column of ones for the intercept) and the targets into a vector y. Training means finding the coefficient vector β that minimises the residual sum of squares.

MODEL

$$\hat{y} = X\beta \quad \Rightarrow \quad \hat{y}_i = \beta_0 + \beta_1 x_{i1} + \ldots + \beta_p x_{ip}$$

Every feature contributes additively. No interaction, no threshold, no curvature unless you add those features yourself.

LOSS - ORDINARY LEAST SQUARES

$$J(\beta) = \Sigma_{i=1}^{n} (y_i - \hat{y}_i)^2 = \|y - X\beta\|^2$$

The squared vertical distance between each point and the plane. OLS is the unique minimiser when **X** *has full column rank.*

Two ways to minimise it:

CLOSED FORM

$\hat{\beta} = (X^T X)^{-1} X^T y$ — exact, but inverting a $p \times p$ matrix costs roughly $O(p^3)$.

GRADIENT DESCENT

$\beta \leftarrow \beta - \eta \cdot (2/n) X^T (X\beta - y)$ — iterative, better when p is large.

WORKED EXAMPLE - THREE POINTS

Data: $(1, 2)$, $(2, 4)$, $(3, 5)$. Means: $\bar{x} = 2$, $\bar{y} = 3.667$.

$S_{xy} = \Sigma(x_i - \bar{x})(y_i - \bar{y}) = 3$, $S_{xx} = \Sigma(x_i - \bar{x})^2 = 2$, so $\beta_1 = 3/2 = 1.5$ and $\beta_0 = 3.667 - 1.5(2) = 0.667$.

Fit: $\hat{y} = 0.667 + 1.5x$. Predictions: 2.167, 3.667, 5.167. Residuals: −0.167, +0.333, −0.167 → **SSE = 0.167**, **R^2 = 1 − 0.167/4.667 = 0.964**. The line explains 96% of the variation — with only three points, do not brag yet.

4a Key terms & the knobs you control — vocabulary and settings

COEFFICIENT	Change in y per one-unit change in x_j, holding the other features fixed.
RESIDUAL	What the model missed: $e_i = y_i - \hat{y}_i$. Its pattern is the diagnostic.
LEVERAGE	How unusual a row's feature values are. High leverage + large residual = influential point.
EXTRAPOLATION	Predicting outside the range of observed x. The line continues; reality may not.

- **Feature engineering** is the real hyper-parameter: polynomial terms, interactions, splines, log targets.
- **Intercept** — keep it unless theory says the line must pass through the origin.
- ***fit_intercept, positive*** — scikit-learn options that constrain the fit.
- **Sample weights** — fit some rows more heavily when they represent more of the population.
- **Scaling** — not needed for OLS, essential for gradient descent and penalised models.

Linear regression is forgiving about *prediction* and strict about *explanation*. These assumptions matter most when you read a coefficient as an effect, less when you only care about the fitted value.

5 ASSUMPTIONS & CONDITIONS

check them, don't hope

> THE MEMORY HOOK: LINE + I
>
> **L**inearity · **I**ndependence · **N**ormality of errors · **E**qual variance — plus **I**ndependent features (no multicollinearity) and no influential points.

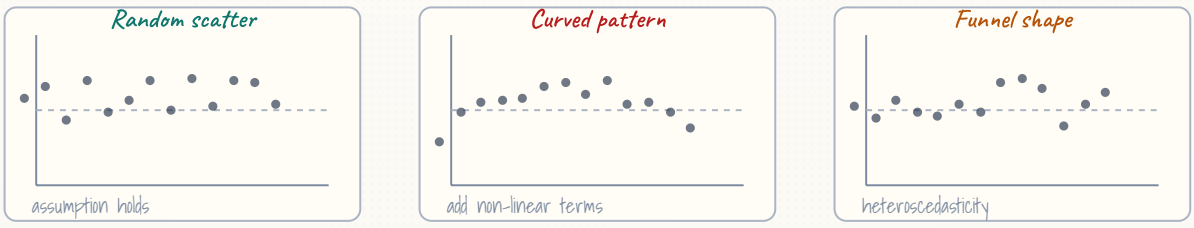
ASSUMPTION	WHAT IT MEANS	HOW IT FAILS	THE FIX
Linearity	*The true relationship is additive and linear in the coefficients.*	*Residuals vs fitted show a curve or fan.*	*Add polynomial terms, splines, interactions, or switch model.*
Independence	*Errors are independent of each other.*	*Time/grouped data: autocorrelated residuals (Durbin-Watson far from 2).*	*Time-series models, clustered standard errors, group-aware splits.*
Normality of errors	*Residuals are roughly $N(0, \sigma^2)$.*	*QQ plot bends away from the line.*	*Needed mainly for CIs and p-values; with large n it matters less.*
Equal variance	*Spread of errors is constant (homoscedasticity).*	*Residual fan or funnel shape.*	*Transform y (log), weighted least squares, robust errors.*
No multicollinearity	*Features are not near-duplicates of each other.*	*VIF > 5–10; huge coefficient standard errors.*	*Drop or combine features; Ridge stabilises the fit.*
No influential outliers	*No single point dictates the fit.*	*Cook's distance > $4/n$.*	*Check the record; use robust regression if genuine.*

5b DIAGNOSTICS THAT ACTUALLY CATCH PROBLEMS

plot before you trust

- **Residuals vs fitted** — should be a shapeless cloud around zero. Shape = wrong model; fan = wrong variance.
- **QQ plot** — points on the diagonal → normal errors (for inference).
- **VIF** = $1/(1-R^2_j)$ per feature; above 5–10 means that coefficient is unstable.
- **Cook's distance** — flag points above 4/n for a second look, not automatic deletion.

> PREDICTION VS INFERENCE
>
> A curved residual plot does not stop the model from ranking rows usefully. But it does invalidate a claim such as "one extra room adds exactly 12,000".



always plot residuals vs. fitted values — the numbers never tell you the whole story

Fig. 10.1 — Three residual plots you will meet in the wild. The numbers can look fine while the shape screams that the model is wrong.

5c A FIVE-MINUTE DIAGNOSTIC RECIPE

ten minutes, once

1. Plot residuals vs fitted: shape, fan, or suspicious points?
2. Plot residuals vs key features, then QQ-plot the tails.
3. Compute VIF for every feature; investigate anything above 5.
4. Refit without high-Cook's-distance rows and compare the slopes.

VARIANCE INFLATION FACTOR

$$VIF_j = 1 / (1 - R_j^2)$$

R_j^2 *is from regressing feature* j *on all the other features. VIF 1 = no overlap; 5–10 = unstable; >10 = the coefficient is hard to interpret.*

6 EVALUATION METRICS — numbers with meaning

Pick the metric before fitting. For regression, the headline number is an error in the target's units; R^2 is context, not a verdict.

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
MAE	$(1/n)\Sigma\lvert y_i - \hat{y}_i\rvert$ — *average miss, same units as y.*	*You want an error a human can read and outliers should not dominate.*
RMSE	$\sqrt{(SSE/n)}$ — *square-root of mean squared error.*	*Large errors are disproportionately costly; the go-to headline.*
R^2	$1 - SSE/SST$ — *share of variance explained.*	*Explaining variation matters and the target is fixed across models.*
Adj. R^2	*Penalises extra features:* $1-(1-R^2)(n-1)/(n-p-1)$.	*Comparing models with different numbers of features.*
MAPE	$(100/n)\Sigma\lvert y_i-\hat{y}_i\rvert/\lvert y_i\rvert$ — *average % error.*	*Business reporting in percentages — never when y is near zero.*

REPORT THE BASELINE TOO

A test RMSE of 18,000 means nothing until you know the mean-only baseline is 24,000. Always print both, plus the fold-to-fold spread.

R^2 TRAPS

It never decreases when you add features, it can be negative on test data, and it says nothing about whether the residuals are well behaved.

READING A REPORT YOU MIGHT ACTUALLY SEE

Test RMSE 18,400 · MAE 12,900 · R^2 0.71 · mean-only baseline RMSE 24,300 · fold SD 900.

The model beats the baseline by about 24%, typical misses are smaller than the RMSE because a few large errors pull it up, and the fold spread is small relative to the gain. That is a model worth keeping — provided the residual plots on page 10 are shapeless.

7 PRACTICAL USE CASES — where it earns its keep

- **House prices:** area, rooms, location dummies and age → price. Coefficient signs are an immediate sanity check.
- **Demand & sales:** price, promotion, seasonality → units sold; the baseline for elasticity questions.
- **Insurance & credit:** claim size or exposure regressed on policy and applicant features.
- **Sensor calibration:** true measurement vs raw reading — the slope is the correction factor.
- **A/B uplift:** regress the outcome on treatment plus pre-period covariates to absorb noise and shrink variance.
- **Feature screening:** t-statistics and standardised coefficients shortlist variables for a heavier model.

REMEMBER

Linear regression is a **description of association**. A large coefficient does not prove that changing the feature changes the outcome — omitted variables, reverse causality and measurement error all hide in that gap.

4b THE LINEAR FAMILY: OLS → RIDGE → LASSO

same line, different leash

When features are many, duplicated or noisy, OLS chases every wobble. Add a penalty on the size of the coefficients and the fit becomes calmer. The predictors stay linear; only the loss changes — which is why these are **still linear regression models**.

PENALISED OBJECTIVES

$$\text{Ridge: } \Sigma(y_i-\hat{y}_i)^2 + \alpha\Sigma\beta_j^2 \quad \cdot \quad \text{Lasso: } \Sigma(y_i-\hat{y}_i)^2 + \alpha\Sigma|\beta_j| \quad \cdot \quad \text{Elastic Net: SSE} + \alpha(\rho\Sigma|\beta_j| + (1-\rho)\Sigma\beta_j^2)$$

$\alpha \geq 0$ *controls the penalty (in scikit-learn:* `alpha`*);* ρ *mixes L1 and L2 (in scikit-learn:* `l1_ratio`*). The intercept is not penalised.*

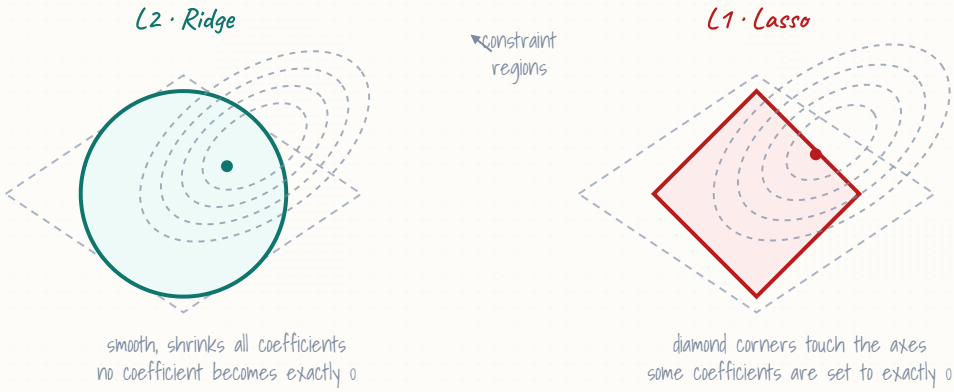


Fig. 1.1 — Why L1 produces zeros: the diamond has corners on the axes, and the loss contours touch a corner first. L2's circle only shrinks coefficients toward zero.

MODEL	PENALTY	EFFECT ON COEFFICIENTS	USE WHEN…
OLS	*none*	*Unbiased, but high variance; unstable with correlated features.*	*Few features, clean data, inference is the goal.*
Ridge	*L2 ·* $\Sigma\beta^2$	*Shrinks smoothly; keeps all features; handles multicollinearity.*	*Many small effects, correlated inputs, prediction focus.*
Lasso	*L1 ·* $\Sigma\|\beta\|$	*Sets some coefficients to exactly zero; selects features.*	*You suspect only a few features matter.*
Elastic Net	*L1 + L2*	*Selection plus grouping of correlated features.*	*Many correlated features; neither pure L1 nor L2 is stable.*

Always standardise features before penalised regression — otherwise the penalty depends on the unit of measurement. Keep the scaler inside the CV pipeline.

4c TUNING THE PENALTY WITHOUT FOOLING YOURSELF

regularisation is a CV decision

- **Tune α by CV**, on a log grid. $\alpha \to 0$ recovers OLS; $\alpha \to \infty$ pushes every slope to zero and leaves the mean.
- **Keep the scaler inside the pipeline**, or the validation fold leaks information into the fit.
- **Do not penalise the intercept** — it would make the model wrong at the average, not just conservative.
- **Correlated features:** Lasso picks one arbitrarily and zeroes the rest; Elastic Net shares the weight more honestly.
- **Coefficients after regularisation are biased** toward zero — fine for prediction, misleading as effect sizes.
- **Report the chosen α** and the CV curve; a single tuned number with no spread is not reproducible.

THE SCALING SHORTCUT

Standardising the whole dataset and then splitting gives the training fold a peek at the test set's mean and variance. It usually looks like a tiny improvement — which is exactly why it survives code review.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES

the honest ledger

STRENGTHS

- ✓ Fast to fit and trivial to explain.
- ✓ Coefficients are effect sizes with CIs.
- ✓ A strong, honest baseline.
- ✓ Works with tiny datasets.
- ✓ Ridge handles correlated features.

LIMITS

- Only linear unless you engineer curvature.
- Sensitive to outliers and influential points.
- Extrapolation is a guess, not a forecast.
- Correlated features split their effect.
- Omitted variables bias the coefficients.

MISTAKES

- Scaling before the train/test split.
- Reading R^2 as proof of a good model.
- Dropping "insignificant" features without checking.
- Comparing coefficients across unscaled features.
- Ignoring residual plots because RMSE looked fine.

THE OUTLIER TRAP

One point at the far end of x can flip a slope from +0.8 to −0.3. Fit with and without suspicious points and report both — never quietly delete the inconvenient ones.

SIGNS IT IS TIME TO MOVE ON

Residuals keep their shape after sensible feature engineering → try a tree model or spline. Many correlated features, prediction is the goal → Ridge. Only a few features matter → Lasso. Effects are clearly non-linear with sharp thresholds → a tree ensemble.

9 SUMMARY

the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · regression · parametric*
Learns	*Coefficients β that minimise squared error*
Decision surface	*A straight line (1-D) or flat hyperplane (p-D)*
Key knob	*Feature engineering; α if regularised*
Report	*RMSE / MAE + R^2 + residual plots + baseline*
Reach for it	*First baseline, effect sizes, small n, explainability*

THE 20-SECOND VERSION

A weighted sum fitted by least squares. It assumes the world is additive and straight. When that holds it is unbeatable value; when it fails, the residual plot tells you before the leaderboard does.

$O(p^3)$

closed-form cost — fine for thousands of features, awkward beyond

√

RMSE is in target units — the easiest number to explain to a stakeholder

PART I · LINEAR MODELS · CHAPTER 02

Logistic Regression

Keep the linear score, then bend it into a probability. A classifier that still speaks the language of odds and coefficients.

SUPERVISED | CLASSIFICATION | PROBABILISTIC | LINEAR BOUNDARY | PAGES 14-17

1 OVERVIEW — what it is

Logistic regression predicts the **probability that a row belongs to a class.** It computes the same weighted sum as linear regression, then passes it through a sigmoid so the output is always between 0 and 1. A threshold — 0.5 by default — turns that probability into a label.

> THE MODEL, IN ONE LINE
>
> $p(y = 1 \mid x) = \sigma(w \cdot x + b) = 1 / (1 + e^{-(w \cdot x + b)})$ — the decision boundary is where $w \cdot x + b = 0$, so it is a straight line, plane or hyperplane.

2 INTRODUCTION & PURPOSE — why it survives

The name is misleading: logistic regression is a **classification** model. It was built for the case where you need more than a hard label — you need a risk, a propensity, a probability that can be ranked and thresholded later.

- **A probability, not just a class.** Compare two patients at 8% and 61% risk, even if both are called "low" today.
- **Interpretable effects.** Coefficients become odds ratios, which regulators and clinicians can argue with.
- **Calibrated by construction.** Maximum likelihood tends to produce probabilities that mean what they say.
- **A stubborn baseline.** On tabular data it is often within a few points of far heavier models.

3 INTUITION — score → odds → probability

Start with a credit score: the linear part says *how many log-odds* of default this applicant carries. A score of 0 means even odds (50%). Positive scores push the probability up, negative scores push it down. The sigmoid is the translator between the unlimited number line and the 0–1 probability scale.

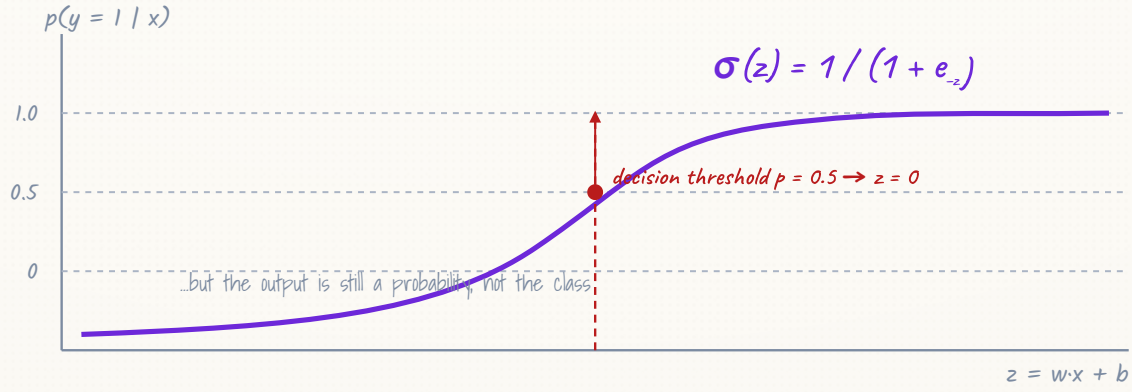


Fig. 14.1 — The sigmoid never reaches 0 or 1, so the model never becomes absolutely certain. The threshold is a business decision layered on top of the probability.

Read it as odds: if $p = 0.80$, the odds are $0.8 / 0.2 = 4$ — four to one in favour. The log-odds, $\ln(4) = 1.386$, is exactly the linear score z. That is why the model is called the *logit* model.

4 How it works — the engine

Two pieces: a linear score, and a link function that maps it to a probability. Training chooses the weights that make the observed labels most likely — **maximum likelihood**, which is exactly minimising log loss.

LINEAR SCORE → PROBABILITY

$$z = w \cdot x + b \qquad p = \sigma(z) \qquad \ln(\, p \,/\, (1-p)\,) = z$$

The log-odds are linear in the features. A one-unit rise in x_j adds w_j to the log-odds, i.e. multiplies the odds by e^{w_j}.

LOG LOSS / BINARY CROSS-ENTROPY

$$J(w) = -(1/n)\, \Sigma_i \, [\, y_i \ln p_i + (1-y_i) \ln(1-p_i) \,]$$

Confident and wrong is expensive: predicting 0.99 for a negative row costs about 4.6. This is the price of a probability, not a label.

The gradient has the same shape as in linear regression — $\nabla J = (1/n) X^T (p - y)$ — but there is no closed form, so solvers iterate (L-BFGS, Newton, SGD).

WORKED EXAMPLE - ONE ROW

Suppose $w = 1.2$, $b = -0.5$, and a row has $x = 2$. Then $z = 1.2(2) - 0.5 = 1.9$, so $p = 1/(1+e^{-1.9}) = 0.87$.

Odds = $0.87/0.13 = 6.7$; log-odds = 1.9 ✓. With a 0.5 threshold the label is positive; with a 0.95 threshold it is negative.

WHY NOT JUST REGRESS ON THE 0/1 LABEL?

A straight line fitted to 0/1 labels can predict 1.4 or −0.2, its errors have variance that depends on p, and squared error optimises the wrong thing. The sigmoid keeps predictions legal; log loss punishes confident mistakes.

4a The knobs and the extensions — raise the ceiling

THRESHOLD	Not learned. Choose it from the precision/recall trade-off your problem can tolerate.
C / λ	Inverse regularisation. Small C = strong L2 penalty = smoother boundary.
CLASS_WEIGHT	Reweights the loss; cheaper than resampling and keeps all rows.
SOLVER	lbfgs for small/medium dense data; saga for L1 or very large data.

- **Multinomial:** replace the sigmoid with a softmax over K classes; the loss becomes categorical cross-entropy.
- **Polynomial features / splines** let a linear-in-parameters model bend its boundary.
- **Perfect separation** sends coefficients to infinity — regularisation is the practical fix.

5 ASSUMPTIONS & CONDITIONS

what must be true

- **Linearity of the log-odds.** Each feature moves the logit by a constant amount. Check with binned log-odds or splines.
- **Independent observations.** Repeated measures per patient need grouped CV or a mixed model.
- **No perfect separation.** A feature that perfectly splits the classes makes the MLE undefined.
- **Enough events per variable.** A rough guide is 10–20 events (minority rows) per feature.
- **Low multicollinearity.** Correlated features make odds ratios unstable, though predictions survive.
- **No extreme outliers** in feature space; they can dominate the likelihood.

THE ASSUMPTION PEOPLE FORGET

Logistic regression assumes the *log-odds* are linear, not the probability and not the features. A feature can have a strong non-linear relationship with risk and still fit badly until you add a spline or a bucket.

6 EVALUATION METRICS

four cells, many questions

Accuracy alone is dangerous: with 2% positives, predicting "negative" forever scores 98%. Always start from the confusion matrix.

THE CONFUSION MATRIX

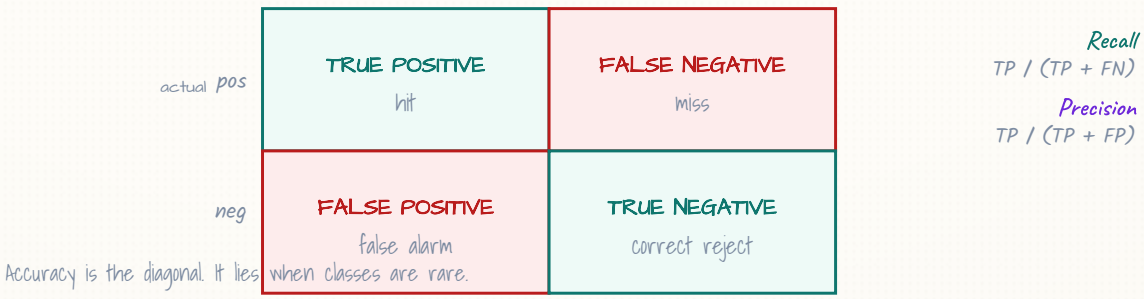


Fig. 16.1 — Four cells, four different questions. Precision asks "when we say positive, how often are we right?"; recall asks "how many true positives did we catch?"

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
Precision	$TP/(TP+FP)$	*False positives are expensive (spam filters, alerts).*
Recall	$TP/(TP+FN)$	*Missing a positive is expensive (disease screening, fraud).*
F1	$2PR/(P+R)$	*You need one balanced number and classes are uneven.*
ROC-AUC	*Ranking quality across all thresholds.*	*Comparing models before choosing a threshold; balanced-ish data.*
PR-AUC	*Precision-recall trade-off, threshold-free.*	*Rare positives — far more honest than ROC here.*
Log loss	$-(1/n)\Sigma[y \ln p + (1-y)\ln(1-p)]$	*The probabilities themselves must be good, not just the ranking.*
Brier	$(1/n)\Sigma(p_i - y_i)^2$	*A mean-squared-error view of probabilistic calibration.*

Specificity = $TN/(TN+FP)$ matters when a false alarm triggers an expensive investigation. Balanced accuracy is the average of recall and specificity — useful under imbalance.

7 PRACTICAL USE CASES

where it earns its keep

- **Credit default & fraud:** probability of default, with odds ratios for the adverse-action letter.
- **Clinical risk:** 30-day readmission or disease risk, where calibration matters more than raw accuracy.
- **Churn & retention:** who is likely to leave, ranked so the retention budget hits the top decile.
- **Marketing response:** propensity to click or convert; the probability feeds expected-value calculations.
- **Spam & abuse:** fast, explainable first line before a heavier model.
- **Epidemiology:** adjusted odds ratios for risk factors — the traditional publishable model.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES

the honest ledger

STRENGTHS

- ✓ Outputs calibrated probabilities.
- ✓ Coefficients are interpretable odds ratios.
- ✓ Fast, stable, works on small n.
- ✓ Regularisation prevents separation blow-ups.
- ✓ A transparent baseline.

LIMITS

- Linear decision boundary by default.
- Needs feature engineering for curves.
- Sensitive to extreme outliers.
- Struggles with rare events and separation.
- Odds ratios are not risk ratios.

MISTAKES

- Reporting accuracy on imbalanced data.
- Keeping the 0.5 threshold by reflex.
- Fitting the scaler before the split.
- Reading e^{β} as a risk multiplier.
- Comparing AUCs from different test sets.

THE THRESHOLD IS A DECISION, NOT A DEFAULT

If a missed fraud costs 20× a false alarm, the optimal threshold is far below 0.5. Report the curve and the chosen operating point — not a single accuracy number.

9 SUMMARY

the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · classification · probabilistic*
Learns	*Weights that maximise the likelihood of the observed labels*
Boundary	*Linear in the features (unless engineered)*
Key knob	*C, threshold, class weights*
Report	*Precision/recall at the chosen threshold, PR-AUC, log loss*
Reach for it	*Probabilities, effect sizes, small/medium tabular data*

LINEAR VS LOGISTIC

Same linear score, different output and loss. Regression predicts a value with squared error; logistic predicts a probability with log loss. If the target is a category, using linear regression is a category error in both senses.

PART 2 · TREE-BASED MODELS · CHAPTER 03

Decision Tree

Twenty questions, asked by a machine: split the data until each group is nearly pure, and read the rules off the branches.

SUPERVISED | CLASSIFICATION & REGRESSION | NON-PARAMETRIC | AXIS-ALIGNED | PAGES 18-21

1 OVERVIEW — what it is

A decision tree partitions the feature space into rectangular boxes and assigns one prediction to each box — a class vote for classification, a mean for regression. The model *is* the set of if/else rules along the branches, which is why it remains the most explainable non-linear learner.

> THE MODEL, IN ONE LINE
>
> Start with all rows at the root; repeatedly ask "is $x_j \le t$?", sending rows left or right; stop when a node is pure, too small, or the depth limit is reached.

2 INTRODUCTION & PURPOSE — why it exists

Trees were designed for decisions people already make with checklists: approve or decline, triage or discharge, flag or ignore. Their purpose is to find those thresholds **automatically** and in the right order, capturing interactions that a linear model would need to be told about.

- **Rules you can defend** — a short path from root to leaf is an auditable reason.
- **Mixed data** — numeric and categorical features, no scaling required.
- **Interactions for free** — "high income *and* young" becomes a single path.
- **Building block** — bagging and boosting are forests of exactly this learner.

3 INTUITION — partition, then predict

Play "guess who". Each question should cut the remaining candidates as unevenly as possible — ideally leaving mostly one class on one side. A good split makes the children purer than the parent; the tree keeps asking until it runs out of useful questions.

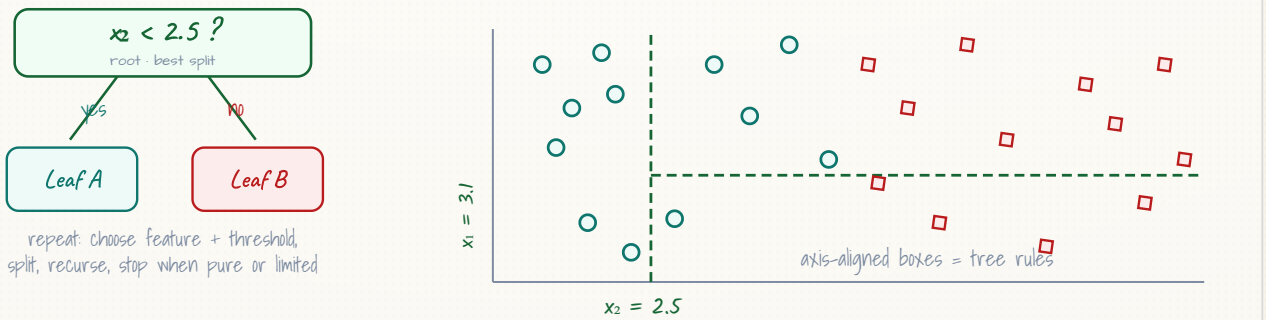


Fig. 18.1 — A tree is a set of nested thresholds. On the right, the same rules drawn in feature space: each split is a horizontal or vertical cut, and each leaf is a rectangle.

Because every cut is parallel to an axis, a diagonal boundary has to be approximated by a staircase. That is why trees need many splits for smooth relationships — and why they shine on tabular data with thresholds and interactions.

4 How it works — the engine

The standard algorithm is **CART**: greedy, recursive, binary splitting. At every node it scans features and thresholds and picks the split that reduces impurity the most — then repeats on each child. Greedy means it never revisits an earlier choice, which is fast but not globally optimal.

IMPURITY - CLASSIFICATION

$$Gini = 1 - \Sigma_k\, p_k^2 \quad \cdot \quad Entropy = -\Sigma_k\, p_k \log_2 p_k$$

p_k = share of class k in the node. Both are 0 for a pure node; Gini is cheaper, entropy is slightly more sensitive to small changes.

INFORMATION GAIN

$$IG = I(parent) - \Sigma_{child}\, (n_{child} / n) \cdot I(child)$$

For regression, replace impurity with variance and the leaf prediction with the mean of its rows: $\hat{y} = (1/n_{leaf})\Sigma y_i$.

1. Start with every training row in the root node.
2. For each feature and each candidate threshold, score the impurity drop.
3. Keep the single best (feature, threshold) pair and split.
4. Recurse on both children — the same feature may be reused.
5. Stop at purity, a size/depth limit, or a negligible gain; then prune.

WORKED EXAMPLE - ONE SPLIT

Parent: 4 positive, 4 negative → Gini = 1 − (0.5² + 0.5²) = **0.50**.

Split into (3+,1−) and (1+,3−). Each child: 1 − (0.75² + 0.25²) = 0.375. Weighted: 0.375.

Gain = 0.50 − 0.375 = 0.125. The best split in the node is the one with the largest gain.

4c Classification and regression trees differ only in the impurity — one algorithm, two tasks

PIECE	CLASSIFICATION TREE	REGRESSION TREE
Split score	*Gini or entropy decrease*	*Variance (MSE) decrease*
Leaf output	*Majority class, or class frequency*	*Mean of the training targets in the leaf*
Prediction shape	*Piecewise-constant class regions*	*Piecewise-constant steps — flat within a box*
Typical metric	*F1, recall, ROC-AUC, log loss*	*RMSE, MAE, R^2*

HOW THE THRESHOLDS ARE ACTUALLY FOUND

For each feature, sort its values and try midpoints between neighbours. That is $O(p \cdot n \log n)$ per node — cheap enough to brute force, which is exactly what CART does.

WHY GREEDY IS STILL USEFUL

A globally optimal tree is NP-hard. The greedy split is fast, usually good, and — crucially — is the same procedure that bagging will later decorrelate into a forest.

4b CONTROLS: GROW, THEN PRUNE

the dials that matter

MAX_DEPTH	Hard ceiling on path length. The first knob to tune.
MIN_SAMPLES_SPLIT	A node must have this many rows before it may split.
MIN_SAMPLES_LEAF	Every leaf must keep this many rows — smooths predictions.
CCP_ALPHA	Cost-complexity pruning: grow fully, then cut back weak branches.
CRITERION	gini / entropy / log_loss for classification; squared_error or absolute_error for regression.
MAX_FEATURES	Limit features considered per split — mainly useful inside ensembles.
CLASS_WEIGHT	Reweight impurity so rare classes are not ignored.
MIN_IMPURITY_DECREASE	Stop when a split buys less than this much purity.

Pre-pruning (depth, leaf size) stops early and is fast but can miss interactions. **Post-pruning** with *ccp_alpha* grows the full tree, then removes branches that do not pay for their complexity — usually the better trade.

5 ASSUMPTIONS & CONDITIONS

what must be true

- **No distribution assumptions** — but rectangular boundaries must suit the problem.
- **Enough rows per leaf**, or the leaf mean/vote is noise.
- **I.i.d. rows.** Grouped or time-ordered data need grouped/time-aware validation.
- **Stable features.** Tiny data changes can reshape the whole tree.
- **In-range prediction.** A tree cannot extrapolate beyond the observed target range.
- **No strong linear trend** that would need dozens of staircase steps.

A TREE IS NOT A PROBABILITY MACHINE

Leaf "probabilities" are just class frequencies in that box — often 0 or 1 with small leaves. Calibrate them, or use the forest, before anyone treats them as risk.

6 EVALUATION METRICS

score the task, inspect the tree

Evaluate the task, not the tree: classification metrics for labels, regression metrics for numbers. Then add tree-specific checks, because a high score from a 40-level tree is usually luck.

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
Accuracy / F1	*Classification correctness; F1 when classes are uneven.*	*Balanced classes (accuracy) or rare positives (F1).*
Precision / recall	*Error costs on each side of the confusion matrix.*	*The business cares more about one kind of mistake.*
ROC-AUC / PR-AUC	*Ranking quality across thresholds.*	*Comparing models before fixing a threshold; PR for rare positives.*
RMSE / MAE	*Regression error in target units.*	*Numeric target; RMSE if big misses hurt most.*
R^2	*Variance explained by the piecewise-constant fit.*	*Same target across candidate models.*
Depth / leaves	*Complexity and interpretability, not accuracy.*	*Deciding whether the tree is explainable enough to ship.*

COST-COMPLEXITY PRUNING

$$R_\alpha(T) = R(T) + \alpha|T|$$

R(T) *= training error of tree T,* ***|T|*** *= number of leaves. Increase* α *until validation error stops improving.*

7 PRACTICAL USE CASES — where it earns its keep

- **Credit & insurance rules:** a short tree doubles as the explanation letter; regulators accept the path.
- **Clinical triage:** vital-sign thresholds in an order a clinician can audit.
- **Churn diagnosis:** which combination of tenure, plan and usage precedes cancellation.
- **Fault diagnosis:** sensor thresholds for machine maintenance, with a human-readable rule set.
- **Feature discovery:** the first few splits reveal interactions worth encoding elsewhere.
- **Missing data:** some implementations route missing values with surrogate splits.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES — the honest ledger

STRENGTHS

- ✓ Interpretable when shallow.
- ✓ Captures non-linearity and interactions.
- ✓ No scaling, handles mixed types.
- ✓ Fast to fit and to explain.
- ✓ Foundation of forests and boosting.

LIMITS

- High variance: small data change, new tree.
- Overfits happily without limits.
- Axis-aligned staircase boundaries.
- Cannot extrapolate.
- Biased toward high-cardinality features.

MISTAKES

- Growing to purity and calling it tuned.
- Pruning using the test set.
- Reading Gini importance from correlated features.
- Treating leaf frequencies as calibrated risk.
- Judging a deep tree by its training accuracy.

THE INSTABILITY DEMONSTRATION

Change one row in a small dataset and rerun. If the root split changes, the "explanation" was never stable. Report the tree, but trust an ensemble or a linear model for the effect size.

9 SUMMARY — the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · classification or regression · non-parametric*
Learns	*Feature/threshold splits that reduce impurity*
Boundary	*Axis-aligned rectangles; piecewise-constant output*
Key knobs	*max_depth, min_samples_leaf, ccp_alpha*
Report	*Task metric + depth/leaf count + CV spread*
Reach for it	*Rules, interactions, mixed tabular data, a base learner*

TREE VS LINEAR

A line says "more x, more y, at a constant rate". A tree says "below this threshold, this; above it, that". Choose the shape that matches the world — or average many trees and stop choosing.

PART 2 · TREE-BASED MODELS · CHAPTER 04

Random Forest

One tree is a nervous expert. Ask hundreds of them, each with a different sample and a different view of the features, and average the answers.

SUPERVISED | CLASSIFICATION & REGRESSION | BAGGING ENSEMBLE | NON-PARAMETRIC | PAGES 22-25

1 OVERVIEW — what it is

A random forest is a collection of decision trees trained on **bootstrapped samples** of the rows, with a **random subset of features** considered at every split. For classification the trees vote (or average their probabilities); for regression the predictions are averaged. The forest is far more stable than any single tree.

> THE MODEL, IN ONE LINE
>
> $\hat{y} = (1/B)\ \Sigma_{b=1}^{B}\ T_b(x)$ for regression, and a majority vote or mean probability for classification — where each T_b is a deep tree grown on a different bootstrap sample.

2 INTRODUCTION & PURPOSE — why it exists

Single trees have low bias and high variance: change a few rows and the tree changes shape. Bagging was invented to fix exactly that. Random forests add a second dose of randomness — a random feature subset at each split — so that the trees make *different* mistakes and their average cancels them.

- **A strong default** for tabular data with little tuning.
- **Non-linear and interaction-rich**, with no scaling or dummy-encoding gymnastics.
- **Built-in validation** through out-of-bag samples.
- **Feature importance** for screening, with caveats on page 24.

3 INTUITION — the wisdom of dissimilar crowds

Ask a hundred analysts to forecast a price. Each sees a different random sample of history and a different random subset of the available signals. Individually they are noisy; averaging their guesses is remarkably stable — provided they are not all making the *same* mistake.

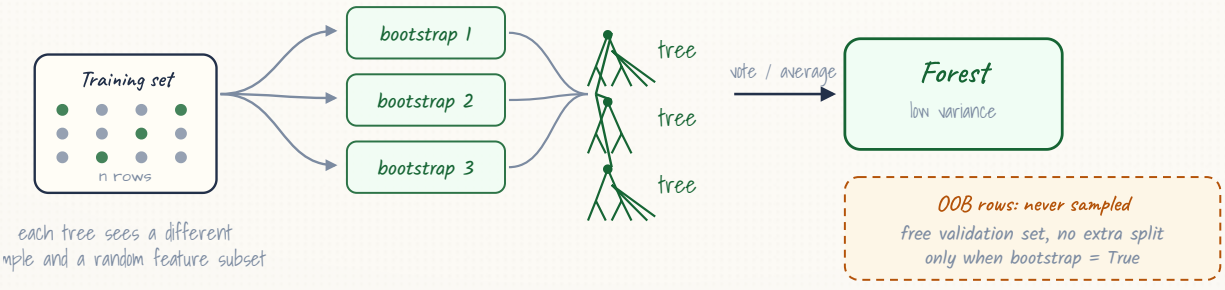


Fig. 22.1 — Draw B bootstrap samples, grow one tree on each, then average or vote. The random feature subset is what stops the trees from becoming copies of one another.

The key word is decorrelation. If every tree saw every feature, they would all split on the same dominant variable and the average would barely help. Forcing different features at each split lowers the correlation between trees — and that is where the variance reduction comes from.

4 How it works — the engine

Bagging plus feature randomness. Each tree is grown deep — usually unpruned — because the averaging, not the individual tree, is responsible for generalisation.

1. For $b = 1 \dots B$, draw a bootstrap sample: n rows with replacement.
2. Grow a tree. At each split, consider only a random subset of m features.
3. Choose the best split among those m (still by Gini or variance reduction).
4. Let the tree grow deep; no pruning by default.
5. Aggregate: majority vote or mean probability / mean prediction.

WHY AVERAGING HELPS

$$Var = \rho\sigma^2 + (1-\rho)\sigma^2 / B$$

ρ = correlation between trees, B = number of trees. More trees shrink the second term; decorrelating them lowers the first — which is why the feature subset matters.

OUT-OF-BAG (OOB) VALIDATION

Each bootstrap sample leaves out about $(1-1/n)^n \to e^{-1} \approx 36.8\%$ of rows. Predict each row using only the trees that never saw it, then score those predictions: a free, nearly-unbiased validation estimate.

TWO KINDS OF IMPORTANCE

MDI / Gini is fast but biased toward high-cardinality and continuous features. **Permutation importance** on held-out data is slower but measures what the model actually uses.

IMPORTANCE IS NOT CAUSATION

A correlated proxy can absorb the importance of the real driver. Check importance stability across folds before you act on it.

4c Where the randomness enters - and what it buys — three sources, one purpose

RANDOMNESS	WHAT IT PREVENTS	WHAT IT COSTS
Bootstrap rows	*Every tree memorising the same outliers and quirks.*	*Each tree sees ≈63% distinct rows; slightly weaker individually.*
Feature subset	*A dominant feature being chosen by every tree, which would leave the trees correlated.*	*Some splits are made without the best feature available.*
Deep, unpruned trees	*Underfitting from stopping too early.*	*High variance per tree — which the averaging then removes.*

MORE TREES DO NOT OVERFIT

Adding trees monotonically stabilises the average; it does not add variance the way adding depth does. The cost is time and memory, not accuracy. 300–500 is usually plenty.

BAGGING VS BOOSTING

Bagging trains trees *in parallel* on perturbed data and averages them to cut variance. Boosting trains them *sequentially*, each fixing the last one's errors, to cut bias. Different tools for different failure modes.

4b HYPER-PARAMETERS WORTH TURNING

defaults are strong

PARAMETER	TYPICAL STARTING POINT	WHAT IT CHANGES
n_estimators	*300–1000*	*More trees → smoother average; diminishing returns, never overfits from count alone.*
max_features	*$\sqrt{p}$ (classification), p (regression)*	*Smaller = more decorrelation, more bias. The single most important knob.*
min_samples_leaf	*1–5*	*Larger leaves smooth noisy targets; useful for regression.*
max_depth	*None*	*Rarely needed; bagging already controls variance.*
bootstrap / oob_score	*True / False*	*OOB scoring requires bootstrap sampling to be meaningful.*
class_weight	*None / balanced*	*Pushes votes toward the rare class; does not fix calibration.*

5 ASSUMPTIONS & CONDITIONS

what must be true

- **Exchangeable rows.** Bootstrap assumes rows are i.i.d.; time series need block bootstrap or time-aware splits.
- **Enough informative features.** If one feature dominates, trees correlate and the forest collapses toward a single tree.
- **Enough rows per tree.** Very small n makes every bootstrap sample a near-duplicate.
- **In-range predictions only.** A forest averages training targets; it cannot extrapolate a trend.
- **Representative sample.** OOB is only as honest as the training data.
- **No free calibration.** Vote fractions are not guaranteed probabilities — calibrate if risk matters.

WHEN NOT TO REACH FOR A FOREST

Images, audio and long sequences are better served by neural networks; smooth extrapolation by linear models; and if you must explain a single coefficient, a forest cannot give you one.

6 EVALUATION METRICS

validation for free

The task decides the metric — but the forest also gives you an honest estimate without a separate validation split.

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
OOB score	*Accuracy or R^2 on out-of-bag rows.*	*A quick sanity check before any tuning; not a replacement for a test set.*
F1 / PR-AUC	*Rare-positive performance.*	*Imbalanced classification; PR-AUC is more honest than ROC.*
ROC-AUC	*Ranking across thresholds.*	*Comparing candidate forests with similar class balance.*
Log loss / Brier	*Quality of the predicted probabilities.*	*Risk scores; requires calibration first.*
RMSE / MAE / R^2	*Regression error and fit.*	*Numeric targets; RMSE if large misses dominate the cost.*
Permutation importance	*Drop in score when a feature is shuffled.*	*Screening features for a simpler model — compute on held-out data.*

THE OOB TRAP

OOB is computed on the training distribution. If your test set comes from a later period or a different segment, OOB will look better than reality. Keep an untouched test set.

7 PRACTICAL USE CASES — where it earns its keep

- **The tabular default:** credit risk, churn, fraud, insurance pricing — strong accuracy with modest tuning.
- **Remote sensing & land cover:** hundreds of spectral bands, non-linear class boundaries.
- **Bioinformatics:** gene-expression classification where p ≫ n and interactions dominate.
- **Demand & sensor forecasting** with rich calendar and lag features (respecting time splits).
- **Feature screening:** permutation importance shortlists variables for a simpler, explainable model.
- **Missing-data imputation:** the forest-style iterative imputer is a strong default for tabular gaps.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES — the honest ledger

STRENGTHS

- ✓ Accurate with little tuning.
- ✓ Robust to outliers and noise.
- ✓ Captures non-linearity and interactions.
- ✓ OOB validation for free.
- ✓ Parallelises across trees.

LIMITS

- Large memory and slow inference.
- No extrapolation beyond training range.
- Less interpretable than one tree.
- Importance can mislead.
- Probabilities need calibration.

MISTAKES

- Using 10 trees and judging the model.
- Tuning depth before max_features.
- Reporting train-set importance.
- Expecting OOB with bootstrap=False.
- Splitting time series at random.

THE LEAKAGE THAT SURVIVES EVERYTHING

A forest will happily memorise a feature that encodes the target — a row ID, a post-outcome field, a duplicate. Check the top of the importance list for anything that could not be known at prediction time.

9 SUMMARY — the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · both tasks · bagged ensemble*
Learns	*Many deep trees on bootstrap samples with random feature subsets*
Boundary	*Smooth average of many axis-aligned partitions*
Key knobs	*n_estimators, max_features, min_samples_leaf*
Report	*Task metric + OOB + permutation importance*
Reach for it	*Tabular data, non-linearity, a strong baseline*

ONE TREE VS A FOREST

The tree is the readable model. The forest is the accurate one. If you need the reason for a single decision, fit a shallow tree or use SHAP — do not read a path out of tree #47 and call it an explanation.

PART 2 · TREE-BASED MODELS · CHAPTER 05

Extra Trees

Extremely randomised trees: don't search for the perfect cut — draw a few at random, keep the best, and move on.

SUPERVISED | CLASSIFICATION & REGRESSION | RANDOMISED ENSEMBLE | FASTER THAN A FOREST | PAGES 26-29

1 OVERVIEW — what it is

Extra Trees (Extremely Randomised Trees) is the sibling of the random forest with the randomness turned up. Splits use **random thresholds** rather than the best possible threshold, and by default each tree is grown on the **whole training set** instead of a bootstrap sample.

> THE MODEL, IN ONE LINE
>
> At each node, pick a random subset of features; for each, draw a threshold uniformly between its min and max; choose the best of those random cuts by impurity. Average many such trees.

2 INTRODUCTION & PURPOSE — why it exists

The forest spends most of its time sorting feature values to find the single best split. Extra Trees asks: what if we skip that search? Random cuts are cheaper, more diverse, and the averaging still works. The result trades a little bias for a further cut in variance and a healthy speed-up.

- **Speed:** no threshold search and no bootstrap copy — often several times faster.
- **More decorrelation:** random cuts make the trees less alike, which helps when a forest's trees are too correlated.
- **Noisy, high-dimensional data** such as spectra, sensors or genomics.
- **A diverse member** of a voting ensemble, precisely because its errors differ from a forest's.

3 INTUITION — randomise, then average

Imagine cutting a cake blindfolded, several times, and keeping the cut that divides the toppings most evenly. You will not find the perfect slice, but you will find a good-enough one very quickly — and each blindfolded baker produces a different cake, so their average is robust.

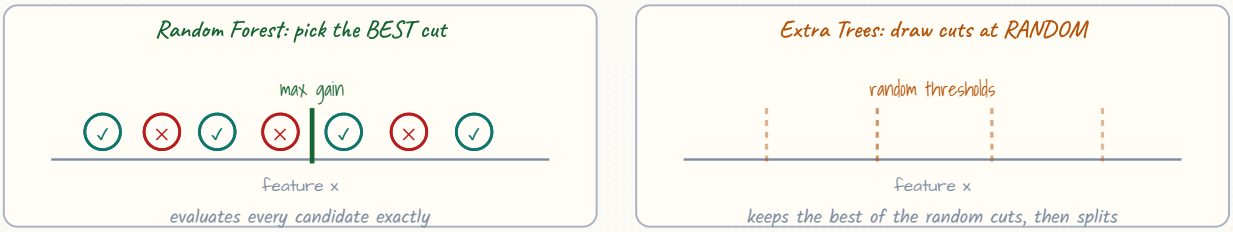


Fig. 26.1 — The forest evaluates every candidate and takes the maximum gain. Extra Trees samples a handful of thresholds and takes the best of those — cheaper, noisier, and more diverse.

The trade: random thresholds increase bias (the cut is not optimal) and reduce variance (trees are less correlated). On many datasets the two roughly cancel; on smooth problems a forest may win, on noisy/high-dimensional ones Extra Trees often matches it while training faster.

4 How it works

the engine

The procedure is the forest with two changes: **no bootstrap** (all rows are used) and **random thresholds** instead of exhaustive search.

1. At each node, draw a random subset of features (size m).
2. For each candidate feature, draw one or more thresholds uniformly between its observed min and max.
3. Score each random cut by impurity; keep the best of the sampled cuts.
4. Split and recurse until leaves are pure or a size limit is hit.
5. Aggregate all trees by vote or mean — no pruning.

SAME AVERAGE, LESS CORRELATION

$$Var = \rho\sigma^2 + (1-\rho)\sigma^2 / B$$

Extra randomisation lowers ρ. That can reduce variance below a forest's — but each tree is individually weaker, so bias rises.

A SPLIT, DRAWN AT RANDOM

Feature X ranges from 10 to 90. A forest sorts all values and tests every midpoint. Extra Trees draws, say, 34 and 61, evaluates both, and keeps 61 because it separates the classes better. It never considers 57 — which is the whole point.

BEHAVIOUR	RANDOM FOREST	EXTRA TREES
Row sampling	*Bootstrap (with replacement)*	*All rows, no bootstrap by default*
Split threshold	*Exhaustive search, best gain*	*Random candidates, best of those*
Bias / variance	*Lower bias, higher correlation*	*Higher bias, lower correlation, often lower variance*
Speed	*Slower (sorting at every node)*	*Faster, especially with many features*
OOB score	*Available by default*	*Only if* `bootstrap=True`

4c Why a random cut can win

bias bought, variance sold

- **No sort, no scan.** The search for the optimal threshold is the expensive part of a tree; deleting it is where the speed comes from.
- **More diversity.** A suboptimal cut changes the whole subtree below it, so trees disagree more — and disagreement is what averaging harvests.
- **Noise robustness.** When the best cut is driven by a few noisy points, a random cut is less likely to chase them.

THE PRICE OF RANDOMNESS

If one precise threshold carries almost all the signal — a regulatory cutoff, a sensor trip point — random cuts may never land near it. A forest or a single well-tuned tree can be materially better.

INCREASE THE TREES

Because each tree is weaker, Extra Trees usually needs more of them than a forest to reach the same stability. Budget 500–1000 for a final model.

4b HYPER-PARAMETERS WORTH TURNING

fewer knobs than the forest

N_ESTIMATORS	300–1000. More trees reduce the variance term; they do not fix a bad bias.	BOOTSTRAP	False by default. Set True only if you want OOB scoring or more row-level diversity.
MAX_FEATURES	1.0 (all) is the scikit-learn default and often best here; lower it for more diversity.	MAX_SAMPLES	Only used with bootstrap=True; controls the draw size per tree.
MIN_SAMPLES_LEAF	Raise it (2–10) when targets are noisy; random cuts can otherwise leave tiny leaves.	MAX_DEPTH	None by default; rarely useful before min_samples_leaf.

TUNE IN THIS ORDER

1) enough trees to stabilise the score, 2) *min_samples_leaf* for noisy targets, 3) *max_features* for the accuracy/speed trade. Depth is the last resort.

5 ASSUMPTIONS & CONDITIONS

what must be true

- **Enough trees to average out the random cuts.** Extra bias means a single tree is even noisier than a single forest tree.
- **Representative rows.** Without bootstrap, every tree sees the full training set; outliers are seen by all of them.
- **Informative features.** Random thresholds can waste cuts on noise variables if *max_features* is large.
- **In-range predictions.** Like every tree ensemble, it cannot extrapolate a trend.
- **Time order matters.** No bootstrap does not mean no leakage — use time-aware splits.
- **Calibration is not automatic.** Vote fractions are scores, not guaranteed probabilities.

6 EVALUATION METRICS

speed is a metric too

Identical metric menu to the forest, with two extra questions: is the speed worth the bias, and is the importance stable?

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
F1 / PR-AUC	*Rare-positive performance.*	*Imbalanced classification; compare against a forest on the same folds.*
ROC-AUC	*Ranking quality.*	*Model comparison when class balance is comparable.*
RMSE / MAE / R^2	*Regression error and fit.*	*Numeric targets — especially with noisy leaves.*
OOB score	*Free validation — only with bootstrap=True.*	*Quick health check; not a substitute for a test set.*
Fit / predict time	*Cost of the extra randomisation.*	*Large feature counts or latency-sensitive serving.*
Permutation importance	*Feature usefulness on held-out data.*	*Screening; MDI importance is noisier here than in a forest.*

DO NOT COMPARE ACROSS DIFFERENT SPLITS

Extra Trees and Random Forest must be judged on the same folds, with the same preprocessing pipeline. A 0.3-point difference is usually noise, not a winner.

7 PRACTICAL USE CASES

where it earns its keep

- **High-dimensional noisy data:** spectra, gene expression, sensor arrays where many features are weak.
- **Fast prototyping:** when a forest is too slow to iterate on, Extra Trees often gets within noise of it.
- **Large tabular problems:** training time matters and the accuracy gap is small.
- **Ensemble diversity:** pair with a forest or gradient-boosted model; different errors make a better vote.
- **Anomaly-adjacent work:** random partitioning is the same family of idea used by isolation-based methods.
- **Baseline for feature screening** before committing to an expensive model.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES

the honest ledger

STRENGTHS

- ✓ Very fast to train.
- ✓ Strong decorrelation between trees.
- ✓ Handles high-dimensional noisy data.
- ✓ Little tuning required.
- ✓ Good ensemble citizen.

LIMITS

- Higher bias than a forest.
- Random cuts can miss the useful threshold.
- Needs more trees for a stable score.
- No OOB unless bootstrap=True.
- Importance is noisy; no extrapolation.

MISTAKES

- Assuming it is identical to Random Forest.
- Expecting OOB with default settings.
- Using 50 trees and declaring a winner.
- Ignoring min_samples_leaf on noisy regression.
- Serving it as a probability without calibration.

THE UNFAIR COMPARISON

"Extra Trees beat the forest" is meaningless if the forest used 100 trees and Extra Trees 1000, or if one was tuned and the other was not. Match the budget and the folds.

9 SUMMARY

the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · both tasks · randomised ensemble*
Learns	*Trees with random feature subsets and random split thresholds*
Boundary	*Average of many rough axis-aligned partitions*
Key knobs	*n_estimators, max_features, min_samples_leaf*
Report	*Task metric + timing + comparison against a forest*
Reach for it	*Speed, noisy high-dimensional tabular data, ensemble diversity*

FOREST OR EXTRA TREES?

Start with a forest: lower bias, OOB built in, the safer default. Try Extra Trees when training time bites, features are many and noisy, or you want a second, differently-wrong model for an ensemble.

PART 3 · DISTANCE & MARGIN MODELS · CHAPTER 06

K-Nearest Neighbours

No equation to fit, no coefficients to read — just look at who lives nearby and let them vote.

SUPERVISED | CLASSIFICATION & REGRESSION | INSTANCE-BASED | LAZY LEARNER | PAGES 30-33

1 OVERVIEW — what it is

K-Nearest Neighbours (KNN) stores the training data and does nothing else. To predict a new row, it measures how far that row is from every stored row, takes the **k closest**, and lets them decide: a majority vote for classification, an average for regression.

> THE MODEL, IN ONE LINE
>
> For a query x, find the set $N_k(x)$ of the k training rows with the smallest distance to x; then predict $\hat{y} = mode\{ y_i : i \in N_k(x) \}$ for a class, or $\hat{y} = (1/k)\Sigma_{i \in N_k} y_i$ for a number.

2 INTRODUCTION & PURPOSE — why it exists

KNN is the machine-learning version of a very old idea: classify something by analogy to the most similar things you have already seen. It needs no training phase, which made it attractive long before compute was cheap — and it is still the honest baseline whenever "similar inputs give similar outputs" is the core assumption.

- **Complex boundaries for free.** No functional form is imposed; the boundary adapts to the local density of each class.
- **Naturally multi-class.** A vote among k neighbours extends to any number of labels without a new model per class.
- **New data without refitting.** Add a row to the store and it is immediately part of every future prediction.
- **A similarity baseline.** If a heavy model cannot beat KNN, the extra complexity is not buying signal.

3 INTUITION — you are the company you keep

You move to a new city and want to guess a neighbour's voting intention. You ask the three households closest to yours. If two of them vote A and one votes B, you predict A. Ask the seven nearest instead and the answer might change — that is the entire role of k.

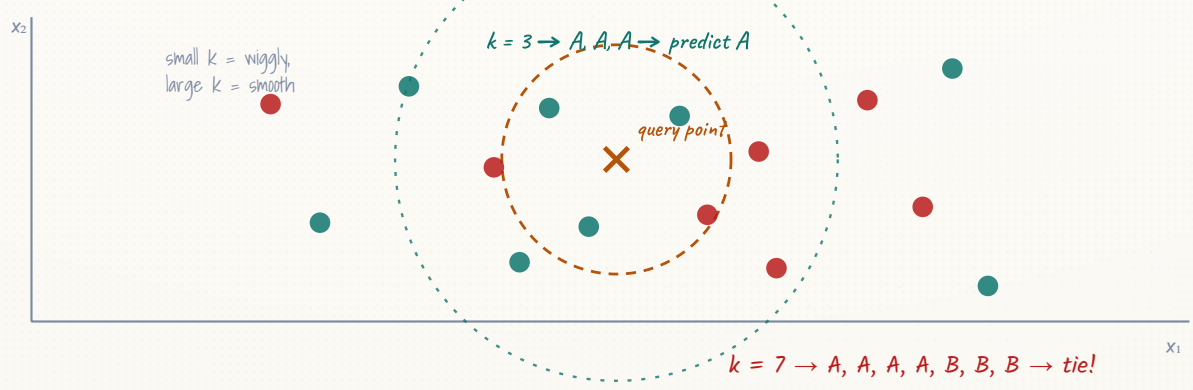


Fig. 30.1 — The query point (the cross) is classified by its nearest labelled rows. With k = 3 the answer is clear; with k = 7 the vote ties, which is why odd values of k are preferred for two-class problems.

Two dials, one model: k controls how local the decision is (small = wiggly, large = smooth), and the **distance metric** decides what "near" even means. Everything else in this chapter follows from those two choices.

4 How it works — the engine

Prediction is a five-step loop, repeated for every query row. There is no parameter-estimation step at all — the "training" phase is just a memory copy.

1. Choose k and a distance measure before you predict.
2. Compute the distance from the query to every stored training row.
3. Sort (or partially select) and keep the k smallest distances.
4. Classification: majority vote — or weight each neighbour by $1/d$.
5. Regression: average the k targets — or take the distance-weighted mean.

WEIGHTED VOTE

$$\hat{y} = \operatorname{argmax}_c \sum_{i \in N_k} w_i \cdot 1[y_i = c] \qquad w_i = 1 / (d_i + \varepsilon)$$

A neighbour at distance 0.2 outvotes one at 2.0 ten times over. The small ε avoids division by zero when the query duplicates a training row.

THE DISTANCES YOU WILL ACTUALLY USE

$$\text{Euclidean: } \sqrt{\Sigma_j (x_j - x'_j)^2} \quad \cdot \quad \text{Manhattan: } \Sigma_j |x_j - x'_j| \quad \cdot \quad \text{Minkowski: } (\Sigma_j |x_j - x'_j|^q)^{1/q} \quad \cdot \quad \text{Cosine: } 1 - (x \cdot x')/(\|x\|\|x'\|)$$

Minkowski with $q = 2$ is Euclidean and $q = 1$ is Manhattan. Cosine ignores magnitude and compares direction — the usual choice for text and embeddings.

WORKED EXAMPLE - THREE NEIGHBOURS, TWO CLASSES

Query distances to the six stored rows: **0.4 (A), 0.9 (B), 1.1 (A), 1.6 (B), 2.2 (A), 3.0 (B)**.

With $k = 3$ the neighbours are A, B, A → vote 2:1 → **predict A**. With distance weights $1/0.4 = 2.50$, $1/0.9 = 1.11$, $1/1.1 = 0.91$, the A score is 3.41 versus 1.11 for B — a much more confident A.

With $k = 5$ the vote is A, B, A, B, A → still A, but by a thinner margin. More neighbours smooth the boundary and blur local detail.

4b Scaling, sparsity and the curse of dimensionality — the three failure modes

DISTANCE IS NOT UNIT-FREE

A salary in rupees contributes about 100,000× more to Euclidean distance than years of experience. **Standardise or min-max scale inside the pipeline**, or the model is effectively using only the largest-unit feature.

TOO MANY DIMENSIONS BREAK THE IDEA OF 'NEAR'

As p grows, the nearest and farthest points become almost equidistant; every neighbour is far away. Feature selection, PCA, or a different model is the fix — not a larger k.

- **Irrelevant features** add noise to every distance. KNN has no built-in way to ignore them.
- **Correlated features** are silently double-counted, so they dominate the metric.
- **Brute force** costs $O(n \cdot p)$ per query — fine for thousands of rows, painful for millions.
- **KD-tree / Ball tree** speed up low-dimensional data; above roughly 20 features they degrade to brute force.

For large n, approximate nearest-neighbour libraries (HNSW, FAISS) trade a little recall for a large speed gain — the standard route for embedding search.

5 ASSUMPTIONS & CONDITIONS

what must be true

- **Comparable scales.** Every feature must be on a scale where a unit difference is meaningful relative to the others.
- **Relevant features.** The distance must reflect similarity in the problem, not just in the data.
- **Local similarity.** Rows close in feature space must tend to share a label or a target value.
- **Enough coverage.** Every region of feature space needs labelled examples; sparse regions get arbitrary answers.
- **No extrapolation.** Regression predictions are averages of stored targets, so they cannot exceed the observed range.
- **Class balance.** A dominant class can swamp the vote unless k is small or neighbours are weighted.

ODD K FOR TWO CLASSES

With two classes and an even k, ties are possible. Odd k removes them for binary problems — but not for multi-class, where ties need an explicit rule.

6 EVALUATION METRICS

the k-curve is the model

Score the task, then score the *choice of k*. The k-curve is the model-selection plot for this family — read the minimum, not the prettiest point.

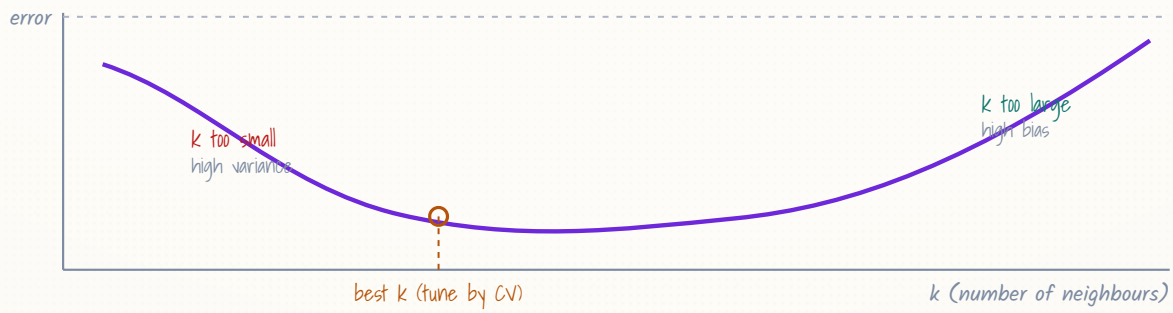


Fig. 32.1 — Validation error against k. Very small k follows noise (high variance), very large k blurs the boundary (high bias). Pick the minimum by cross-validation, then check that the curve is flat around it.

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
Accuracy / F1	*Correctness of the vote; F1 when labels are uneven.*	*Balanced classes (accuracy) or rare positives (F1).*
Precision / recall	*Which kind of mistake the neighbours make.*	*One error is materially more expensive than the other.*
ROC-AUC / PR-AUC	*Ranking quality across thresholds (use predict_proba).*	*Comparing k values or distance metrics before fixing a threshold.*
RMSE / MAE	*Regression error in target units.*	*Numeric target — RMSE when large misses matter most.*
R^2	*Share of variance explained by the local average.*	*Same target across candidate values of k.*
k and metric	*The two hyper-parameters that define the model.*	*Always report them; a score without k is not reproducible.*

NEVER SCORE ON THE TRAINING ROWS

With $k = 1$, every training row is its own nearest neighbour, so training accuracy is 100% by construction. Use a held-out split or cross-validation, always.

6b CHOOSING K WITHOUT GUESSING

tune it like any hyper-parameter

1. Standardise features inside a pipeline.
2. Search k on a grid (1, 3, 5, 7, 9, 15, 25, 50…) with the same folds as every other model.
3. Compare distance metrics only once k is sensible — the two interact.

- **k = 1** chases every outlier; **k ≈ √n** is a search starting point, not a rule.
- **Large k** drifts toward the global majority or mean — high bias, no local detail.
- Report the CV curve, the chosen k, and the fold spread.

7 PRACTICAL USE CASES

where it earns its keep

- **Recommendation:** "users who liked this also liked…" is item-item nearest neighbours on embeddings.
- **Image & document retrieval:** nearest neighbour search in a learned embedding space.
- **Imputation:** fill a missing value from the k most similar complete rows — the KNN imputer.
- **Anomaly detection:** distance to the k-th neighbour is a simple, explainable novelty score.
- **Handwriting & character recognition:** the classic MNIST baseline, still a useful check.
- **Medical & fault diagnosis:** "the three most similar past cases were…" is a natural explanation.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES

the honest ledger

STRENGTHS

- ✓ No training phase; instant to "fit".
- ✓ Arbitrarily complex decision boundaries.
- ✓ Multi-class by construction.
- ✓ New rows can be added without refitting.
- ✓ Predictions are easy to illustrate with the actual neighbours.

LIMITS

- Slow and memory-hungry at prediction time.
- Sensitive to feature scaling and irrelevant features.
- Degrades badly in high dimensions.
- No global model, coefficients or extrapolation.
- Imbalanced data skews the vote.

MISTAKES

- Scaling before the train/test split.
- Picking k because it looks round.
- Even k on a two-class problem.
- Euclidean distance on raw, unscaled features.
- Evaluating with $k = 1$ on the training set.

THE TIME-SERIES TRAP

In ordered data the nearest row in feature space is often the *next* row in time — a near-duplicate that leaks the answer. Use time-aware splits and lagged features, never random shuffling.

9 SUMMARY

the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · both tasks · instance-based, non-parametric*
Learns	*Nothing — it stores the training rows*
Boundary	*Locally adaptive; a patchwork of neighbourhoods*
Key knobs	*k, distance metric, weights, scaling*
Report	*Task metric at the chosen k + the CV curve*
Reach for it	*Similarity search, small low-dimensional data, a quick baseline*

KNN IN ONE SENTENCE

Predict from the k most similar stored rows — which makes the distance metric the model, the scaling a correctness requirement, and the curse of dimensionality the reason it fails quietly on wide data.

PART 3 · DISTANCE & MARGIN MODELS · CHAPTER 07

Support Vector Machine

Many lines can separate two classes. The SVM insists on the one with the widest empty street — then bends space when no straight street exists.

SUPERVISED | CLASSIFICATION & REGRESSION | MARGIN-BASED | KERNEL METHODS | PAGES 34-38

1 OVERVIEW

what it is

A support vector machine finds the decision boundary that is **as far as possible from both classes**. Only a handful of training rows — the *support vectors* — determine that boundary; everything else could be deleted without changing it. Kernels let the same machinery draw curved boundaries in the original feature space.

> THE MODEL, IN ONE LINE
>
> Find w and b minimising $\tfrac{1}{2}\|w\|^2 + C\Sigma\xi_i$ subject to $y_i(w\cdot x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$ — a wide margin, with a budget C for rows that refuse to stay on their side.

2 INTRODUCTION & PURPOSE

why it exists

Developed by Vladimir Vapnik and colleagues in the 1990s, the SVM was the dominant classifier before deep learning — especially where **rows are few and features are many**. Its purpose is to generalise well by controlling capacity: a boundary with a wide margin is less likely to be an accident of the training sample.

- **Small n, wide p.** Text and genomics, where a linear boundary in a huge feature space is already expressive.
- **Clear margins.** When classes are genuinely separable, the max-margin answer is stable and unique.
- **Kernels instead of feature engineering.** A similarity function can encode curvature without you writing the transformed features.
- **Compact model.** Only support vectors are stored, so prediction does not scan the whole training set.

3 INTUITION

the widest street

Two neighbourhoods face each other across a street. Countless lines could divide them, but the safest road is the widest one that touches neither side: the houses on each edge define the street, and everything behind them is irrelevant. Those edge houses are the support vectors.

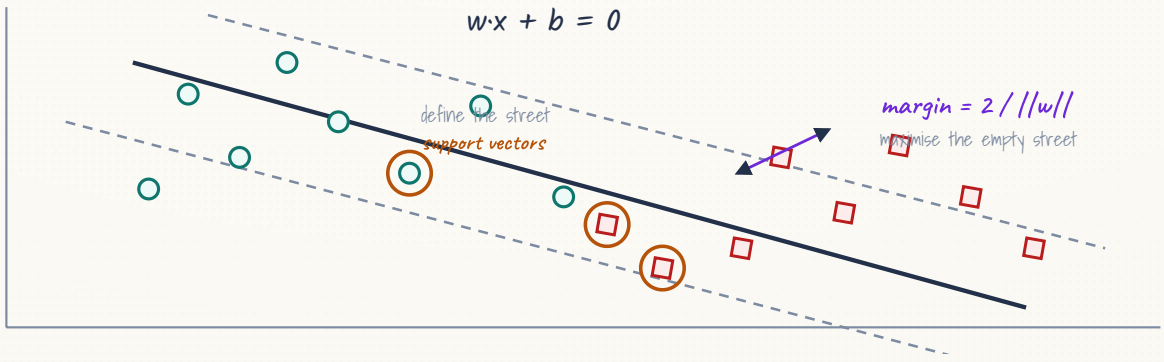


Fig. 34.1 — Maximise the empty band between the classes. The circled points sit exactly on the margin and alone determine the boundary; moving an interior point does not move the line.

Why widest is best: a wide margin means small perturbations of a test point rarely flip its side. That is a geometric argument for robustness — the same reason a fence built down the middle of a wide street is safer than one hugging a house.

4 How it works — the engine

A linear SVM is a hyperplane plus a margin. Training is a constrained optimisation problem; the solution is expressed through a small set of support vectors.

Hyperplane and decision

$$w \cdot x + b = 0 \quad \Rightarrow \quad f(x) = \text{sign}(w \cdot x + b) \qquad \text{distance} = |w \cdot x + b| \,/\, \|w\|$$

*The vector **w** points orthogonal to the boundary; **b** shifts it. Scaling **w** and **b** together changes nothing about the plane — only the margin formula fixes the scale.*

Canonical margin

$$y_i(w \cdot x_i + b) \geq 1 \quad \Rightarrow \quad \text{margin} = 2 \,/\, \|w\|$$

Requiring the closest points to sit at functional distance 1 turns "widest street" into "smallest $\|w\|$". Maximising the margin is minimising $\frac{1}{2}\|w\|^2$.

Soft margin - the real objective

$$\min_{w,b,\xi} \tfrac{1}{2}\|w\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

*ξ_i is how far row i is allowed inside the margin or on the wrong side. **C** prices that violation: large C → few violations, narrow margin; small C → wide margin, more tolerance.*

Equivalent hinge-loss form

$$\min \tfrac{1}{2}\|w\|^2 + C \sum_i \max(0,\, 1 - y_i(w \cdot x_i + b))$$

Zero loss once a point is correct and outside the margin; a linear penalty for every unit it intrudes. This is the form scikit-learn's `SVC(kernel='linear')` optimises.

Worked example - two points, one dimension

Rows: $x = -1$ (class −1) and $x = +1$ (class +1). By symmetry $b = 0$; the constraints give $w = 1$.

Margin $= 2/\|w\| = 2$ — exactly the gap between the points. Move them to −2 and +2 and the optimal $w = 0.5$, margin 4: the street widens because the classes are further apart.

4b Support vectors and the dual — most rows do not matter

- The Lagrangian solution gives one multiplier α_i per training row.
- $\alpha_i = 0$ → the row is irrelevant; delete it and nothing changes.
- $\alpha_i > 0$ → a support vector: on the margin, or inside it when $\alpha_i = C$.

Dual prediction form

$$f(x) = \sum_i \alpha_i \, y_i \, K(x_i, x) + b$$

Only dot products of rows appear — which is exactly what makes the kernel trick possible. The sum runs over support vectors, not all rows.

Why the model is compact

If 4% of rows end up as support vectors, prediction touches 4% of the data. That is why an SVM can be memory-efficient at inference even when n is large — though training still scales roughly $O(n^2)$ to $O(n^3)$.

4c KERNELS: CURVES WITHOUT WRITING THE TRANSFORMATION

similarity replaces coordinates

If no straight boundary works, map the features into a richer space where one does. The kernel trick computes the inner product there directly, without ever building the coordinates.

THE KERNEL TRICK

$$K(x, x') = \phi(x) \cdot \phi(x') \qquad f(x) = \Sigma_i\, \alpha_i\, y_i\, K(x_i, x) + b$$

*Replace every dot product in the dual with **K**. The optimisation is unchanged; only the similarity notion changes.*

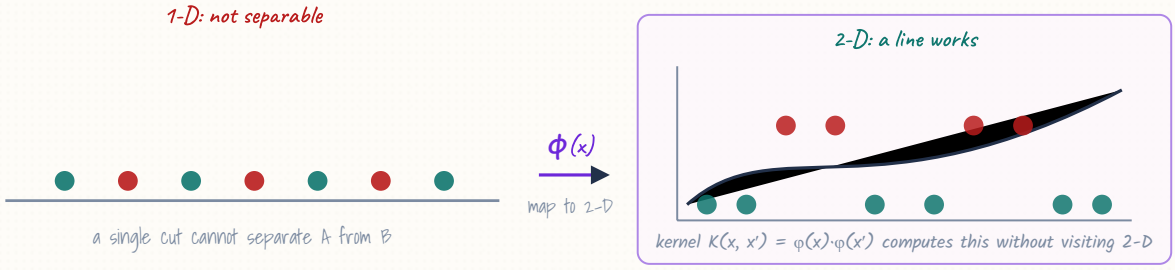


Fig. 36.1 – Left: one feature cannot separate the classes. Right: after a mapping φ, a straight line does the job – and the kernel computes the 2-D inner product without visiting 2-D.

KERNEL	K(x, x')	USE WHEN…
Linear	$x \cdot x'$	*p is large and sparse (text, tf-idf); a linear boundary is already strong.*
Polynomial	$(\gamma\, x \cdot x' + r)^d$	*Interactions up to degree d matter; d is small (2–3).*
RBF / Gaussian	$\exp(-\gamma \lVert x - x' \rVert^2)$	*The default for dense, low-to-medium p data with smooth curved boundaries.*
Sigmoid	$\tanh(\gamma\, x \cdot x' + r)$	*Rarely the best choice; historically linked to neural networks.*

4d C, GAMMA AND THE SHAPE OF THE BOUNDARY

two dials, one trade-off

LARGE C	Few violations allowed: narrow margin, complex boundary, overfitting risk.	LARGE GAMMA	Each point has a local reach: wiggly boundary, overfitting risk.
SMALL C	Wide margin, many tolerated violations: smoother, underfitting risk.	SMALL GAMMA	Far reach: almost linear, smooth, underfitting risk.

TUNING THEM TOGETHER

Search C and γ on a log grid – typically $C \in \{\{0.01 \ldots 1000\}\}$ and $\gamma \in \{\{0.001 \ldots 10\}\}$ – with the same folds for every pair. A huge γ on unscaled features makes every point look isolated, and the model memorises the training set.

4e SUPPORT VECTOR REGRESSION (SVR)

the regression form

SVR inverts the idea: fit a tube of width $\pm\varepsilon$ around the predictions and ignore errors inside it. Only rows outside the tube become support vectors and pay a penalty.

E-INSENSITIVE LOSS

$$L_\varepsilon = \max(0, \lvert y - f(x) \rvert - \varepsilon) \qquad \min \tfrac{1}{2}\lVert w \rVert^2 + C\, \Sigma_i\, L_\varepsilon$$

***ε** sets how much error is free; **C** prices the rest. Large ε → fewer support vectors, flatter fit; small ε → a tighter, more flexible fit.*

5 ASSUMPTIONS & CONDITIONS

what must be true

- **Meaningful similarity.** The kernel must reflect what makes two rows alike in the problem, not just in the units.
- **Comparable scales.** Standardise features; RBF and polynomial kernels are distance- and dot-product-based.
- **Tunable capacity.** C and γ must be selected by cross-validation, never left at a lucky default.
- **Roughly separable classes, or a soft margin.** Heavy label noise needs a smaller C and a simpler kernel.
- **Class balance, or explicit weights.** `class_weight='balanced'` is cheaper than resampling.
- **Manageable n.** Kernel training is roughly $O(n^2)$–$O(n^3)$; beyond ~100k rows consider linear SVM or another model.

PROBABILITIES ARE AN ADD-ON

`SVC(probability=True)` fits an internal cross-validated calibration (Platt scaling), which is slow and can disagree with the decision function's ranking. If you need ranked risk, use `decision_function` or calibrate separately.

6 EVALUATION METRICS

score the task, inspect the margin

Same classification menu as any classifier, with one SVM-specific habit: check the margin distribution, not only the labels.

METRIC	WHAT IT TELLS YOU	REACH FOR IT WHEN…
Precision / recall	*Error cost on each side of the confusion matrix.*	*One kind of mistake is materially worse — pick the threshold deliberately.*
F1	*Balance of precision and recall.*	*Uneven classes and you need a single headline.*
ROC-AUC	*Ranking quality across thresholds.*	*Comparing kernels or (C, gamma) before fixing an operating point.*
PR-AUC	*Ranking quality when positives are rare.*	*Fraud, disease, defect detection — ROC is too optimistic.*
Balanced accuracy	*Mean of recall and specificity.*	*Classes are imbalanced and both errors count.*
Log loss / Brier	*Probability quality, after calibration.*	*Scores feed a risk model; needs* `probability=True` *or a separate calibrator.*
RMSE / MAE / R^2	*SVR error in target units.*	*Numeric target; RMSE when large misses dominate the cost.*

READ THE DECISION VALUES

`decision_function` returns signed distance from the hyperplane. Rows near zero are the uncertain ones — useful for manual review even when the label is confident-looking.

6b A FAIR COMPARISON WITH THE OTHER MODELS

different shapes, different strengths

QUESTION	SVM (RBF)	RANDOM FOREST
Best when	*Small/medium n, many features, smooth boundary*	*Large n, mixed tabular data, interactions*
Scaling	*Mandatory*	*Not needed*
Interpretability	*Low — support vectors, not coefficients*	*Medium — importance plus partial dependence*
Prediction cost	*Grows with the number of support vectors*	*Grows with the number of trees*
Failure mode	*Bad kernel or untuned* γ	*Leakage and over-trusted importance*

7 PRACTICAL USE CASES

where it earns its keep

- **Text classification:** linear SVM on tf-idf features is a famously strong baseline for spam and topic labels.
- **Image classification (pre-deep-learning):** HOG or SIFT features + SVM; still a solid small-data baseline.
- **Bioinformatics:** microarray and gene-expression data where p ≫ n and the boundary is roughly linear.
- **Face and object detection:** sliding-window SVM classifiers were the standard before CNNs.
- **Anomaly detection:** one-class SVM learns a boundary around normal data.
- **Small-data regression:** SVR with an RBF kernel when n is small and smoothness is a reasonable prior.

8 ADVANTAGES, LIMITATIONS & COMMON MISTAKES

the honest ledger

STRENGTHS

- ✓ Effective in high-dimensional spaces.
- ✓ Memory-efficient: only support vectors are stored.
- ✓ Kernels handle non-linearity without manual features.
- ✓ Strong theory and stable margins.
- ✓ Works when n is small but p is large.

LIMITS

- Slow training beyond ~100k rows.
- Sensitive to scaling and kernel choice.
- No native probabilities; calibration costs time.
- Hard to interpret beyond support vectors.
- Multi-class needs one-vs-one or one-vs-rest.

MISTAKES

- Fitting without standardising.
- Tuning C and gamma on the test set.
- Leaving gamma at 'scale' on unscaled data.
- Using RBF on very sparse text by reflex.
- Trusting probability=True on a huge dataset.

THE KERNEL MISMATCH

An RBF kernel on 50,000 sparse text features is usually worse and far slower than a linear kernel. Match the kernel to the geometry of the data, not to its reputation.

9 SUMMARY

the card to revise

QUESTION	SHORT ANSWER
Type	*Supervised · classification (SVC) or regression (SVR) · margin-based*
Learns	*Maximum-margin hyperplane, expressed through support vectors*
Boundary	*Linear by default; curved with a kernel*
Key knobs	*kernel, C, gamma, epsilon (SVR)*
Report	*Precision/recall at the chosen threshold, PR-AUC, calibration if risk*
Reach for it	*Small n with many features, clear margins, text or omics*

SVM IN ONE SENTENCE

Choose the boundary with the widest margin, let a few support vectors define it, and use a kernel when the data needs a curve — but scale the features first, because the whole model is geometry.

PART 4 · THE REFERENCE SHELF · R1

Regression Metrics

One number can never describe a model that predicts quantities. Pick an error, report a baseline, and look at the residuals.

REFERENCE | REGRESSION | PAGE 39

1 THE METRICS, WITH THEIR HONEST USE

eight numbers, one habit

METRIC	FORMULA / MEANING	REACH FOR IT WHEN…
MAE	$(1/n)\Sigma\lvert y_i - \hat{y}_i\rvert$	*You want an error in target units that a human can read; outliers should not dominate.*
MSE	$(1/n)\Sigma(y_i - \hat{y}_i)^2$	*You are optimising or comparing losses mathematically; units are squared, so it is not a headline.*
RMSE	$\sqrt{(1/n)\Sigma(y_i - \hat{y}_i)^2}$	*Large errors are disproportionately expensive; the standard headline for regression.*
MedAE	$\text{median}(\lvert y_i - \hat{y}_i\rvert)$	*The error distribution is skewed or heavy-tailed; you want the typical miss.*
R^2	$1 - SSE / SST$	*You need the share of variance explained, on a fixed target, for context beside an error.*
Adj. R^2	$1 - (1-R^2)(n-1)/(n-p-1)$	*Comparing models with different numbers of features on the same data.*
MAPE	$(100/n)\Sigma\lvert y_i-\hat{y}_i\rvert / \lvert y_i\rvert$	*Business reporting in percentages — never when the target is zero or near zero.*
RMSLE	$\sqrt{(1/n)\Sigma(\ln(1+y_i) - \ln(1+\hat{y}_i))^2}$	*Targets span orders of magnitude (counts, prices); under-prediction is penalised more gently.*

Bold the one metric you will be judged on, and always print the mean-only baseline beside it. A model with RMSE 18,000 is good or bad only relative to a baseline of 24,000 or 17,500.

2 READING THE NUMBERS TOGETHER

context is part of the metric

- **RMSE > MAE** means a handful of large misses are inflating the error — inspect those rows before celebrating.
- **MAE ≈ RMSE** means the errors are fairly uniform; the model fails consistently rather than dramatically.
- **R^2 can be negative** on test data. It means the model is worse than predicting the test mean.
- **Compare against a baseline** (mean, last value, a simple rule) and report the fold-to-fold spread, not just the mean.
- **Break the error down** by segment: a good aggregate can hide a terrible subgroup.
- **Report units.** An RMSE of 18 is meaningless without "thousands of rupees".

3 THE RESIDUAL CHECKS THAT EXPOSE A BAD FIT

the plot is the verdict

1. Plot residuals vs fitted values: a curve means the functional form is wrong; a fan means variance is not constant.
2. Plot residuals vs each important feature: missed thresholds and curvature appear here.
3. QQ-plot the residuals, and inspect the tails rather than every wiggle.
4. Check the worst 10 errors by hand. Are they data errors, rare segments, or genuine model failure?

A REPORT YOU CAN DEFEND

RMSE 18,400 · **MAE** 12,900 · **R^2** 0.71

Baseline RMSE 24,300 · fold SD 900

24% better than baseline, a typical miss below the RMSE, and a tight fold spread. Worth keeping — if the residual plots are shapeless.

PART 4 · THE REFERENCE SHELF · R2

Classification Metrics

Accuracy is a single cell of a four-cell table. Choose the cell that matches what a mistake costs.

REFERENCE | CLASSIFICATION | PAGE 40

THE CONFUSION MATRIX

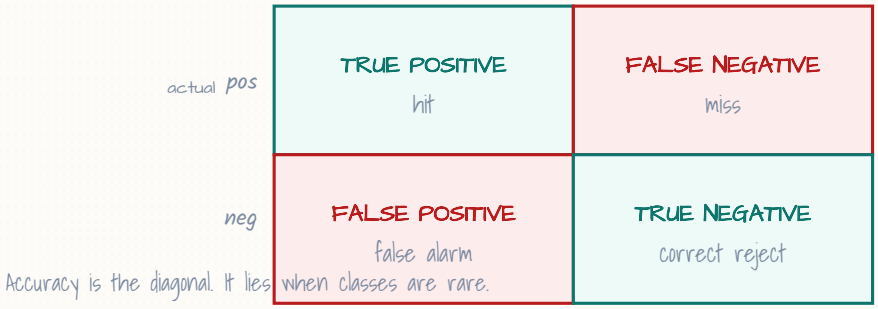


Fig. 40.1 — Every classification metric is a ratio built from these four cells. Row = truth, column = prediction.

1 THE METRICS, BY THE QUESTION THEY ANSWER

pick by cost of error

METRIC	FORMULA	ANSWERS…
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	*How often is the model right overall? Misleading when classes are rare.*
Precision	$TP/(TP+FP)$	*When it says positive, how often is it right? Costly false alarms.*
Recall	$TP/(TP+FN)$	*Of the real positives, how many were caught? Costly misses.*
Specificity	$TN/(TN+FP)$	*Of the real negatives, how many were correctly cleared?*
F1	$2PR/(P+R)$	*One balanced number when classes are uneven.*
Balanced acc.	$(recall + specificity)/2$	*Both classes matter equally under imbalance.*
ROC-AUC	$\int TPR\ d(FPR)$	*Ranking quality across all thresholds, balanced-ish data.*
PR-AUC	$\int precision\ d(recall)$	*Ranking quality when positives are rare — the honest ROC alternative.*
Log loss	$-(1/n)\Sigma[y \ln p + (1-y) \ln(1-p)]$	*Are the probabilities themselves good? Punishes confident errors.*
Brier	$(1/n)\Sigma(p_i - y_i)^2$	*Probability quality as a mean squared error.*

2 MATCH THE METRIC TO THE COST

start from the mistake, not the metric

- **A missed fraud costs more than a review.** Maximise recall, report precision at that recall, and accept more false alarms.
- **A false alarm costs a customer.** Maximise precision, then check recall is still tolerable for the business.
- **Both mistakes cost about the same.** F1 or balanced accuracy, with the confusion matrix printed beside it.
- **You must rank, not label.** Report PR-AUC (rare positives) or ROC-AUC (balanced), then choose a threshold separately.
- **The probability is the product.** Report log loss or Brier score, and add a reliability curve.
- **Classes are rare.** Accuracy is a trap: 99% is what "always predict negative" scores.

Report **precision and recall at the chosen threshold**, plus **PR-AUC** for ranking, and add the confusion matrix so the trade-off is visible.

PART 4 · THE REFERENCE SHELF · R3

Curves, Thresholds & Calibration

A classifier outputs a score. Turning that score into a decision is a separate choice — and it is usually the one that costs money.

REFERENCE | OPERATING POINT | PAGE 41

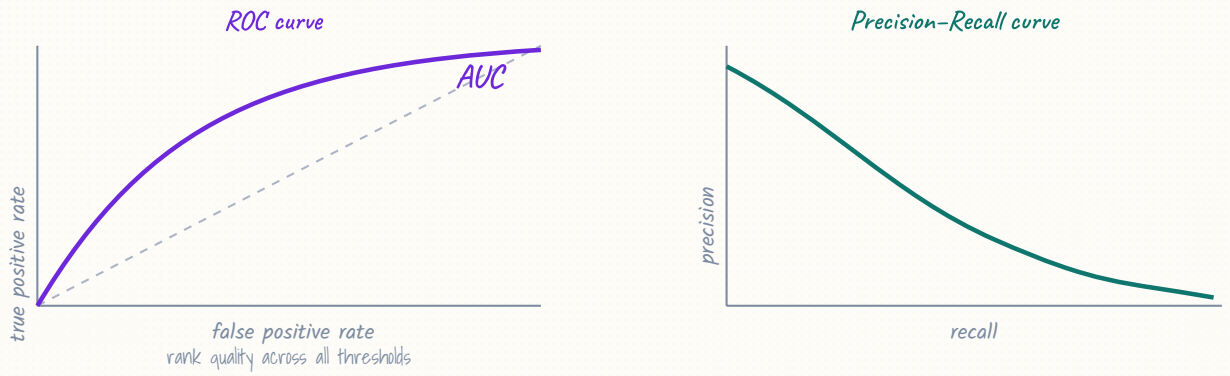


Fig. 41 — ROC asks how well the scores rank across all thresholds; the PR curve is the same question when the positive class is rare, and it is far less flattering.

1 ROC OR PR? ASK HOW RARE THE POSITIVE IS

the curve is a question, not a score

CURVE	AXES	READS AS…	USE WHEN…
ROC	*TPR vs FPR*	*Probability a random positive outranks a random negative*	*Classes are roughly balanced, or both errors matter equally.*
PR	*Precision vs recall*	*How clean the positive predictions are at each recall level*	*Positives are rare — fraud, disease, defects.*
Lift	*Lift vs share targeted*	*How much better than random at the top of the ranking*	*The business acts only on the top 1–5% of scores.*
Calibration	*Predicted vs observed rate*	*Whether a 0.7 really happens 70% of the time*	*The probability itself is used — pricing, risk, triage.*

A model can rank perfectly (AUC = 1) and still be badly calibrated — ranking and probability are different promises. Report the one the decision actually needs.

2 CHOOSING THE THRESHOLD

a business decision in a formula

- **0.5 is a default, not a decision.** It is optimal only when both errors cost the same and the classes are balanced.
- **Set it from costs.** Expected cost = $C_{FP} \cdot FP + C_{FN} \cdot FN$; pick the threshold that minimises it on validation folds.
- **Constrain the operation.** "Review at most 200 cases a day" fixes a capacity, which fixes the threshold.
- **Re-check after deployment.** The best threshold drifts with prevalence, so monitor the positive rate, not just the AUC.

THE TWO-NUMBER HABIT

Report **precision and recall at the chosen threshold**, plus **PR-AUC** for ranking. Add log loss only if the probability itself drives a downstream decision.

PART 4 · THE REFERENCE SHELF · R4

Calibration & the Cost of a Mistake

A ranking model is not a risk model. These are the last two checks before a classifier is allowed to make decisions.

REFERENCE | CALIBRATION | PAGE 42

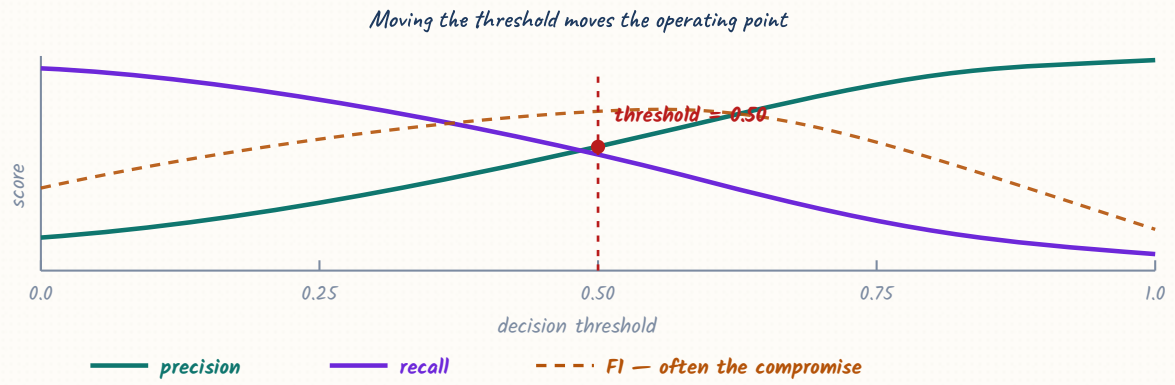


Fig. 42.1 — Moving the threshold trades precision against recall. The best point is where the marginal cost of another false negative equals the marginal cost of another false positive.

2 CHOOSE THE OPERATING POINT FROM THE COST

the table the business should see

- **Write the costs down.** A missed fraud at 10× a false alarm is a different model from a 1:1 problem.
- **Score each candidate threshold.** Expected cost = $C_{FP} \cdot FP + C_{FN} \cdot FN$, on validation folds only.
- **Constrain by capacity.** "Review 200 cases a day" fixes the threshold without any cost model at all.
- **Re-check after deployment.** Prevalence drifts; monitor the positive rate, not only the AUC.

THRESHOLD	TP	FP	FN	PRECISION	COST AT 10:1
0.30	46	74	4	0.38	10×4 + 74 = **114**
0.50	30	90	20	0.25	10×20 + 90 = **290**
0.70	14	21	36	0.40	10×36 + 21 = **381**

READ THE BOTTOM LINE

Same model, same scores, three operating points. Only the costs decide — and here the lower threshold wins despite its terrible precision.

3 CALIBRATION - WHEN THE NUMBER ITSELF MATTERS

probabilities are a separate promise

- **Reliability curve.** Bin the predictions and plot predicted rate against observed rate; the diagonal is perfect calibration.
- **Platt scaling.** Fit a one-dimensional logistic regression on held-out scores; cheap and smooth, good for SVMs.
- **Isotonic regression.** A monotone step fit; more flexible, needs more data, can overfit a small calibration set.

NEVER CALIBRATE ON THE TEST SET

Fit the calibrator on a held-out slice or inside cross-validation. Calibrating on the data used to choose the threshold guarantees an optimistic curve.

ONE PROMISE PER SENTENCE

Ranking: "the top 5% finds 4× more fraud". **Operating point:** "we catch 60%, reviewing 120 cases". **Probability:** "when we say 30%, it happens 28% of the time".

PART 4 · THE REFERENCE SHELF · R5

Clustering Metrics

There is no answer key, so "accuracy" does not exist. Judge the shape of the groups, and whether they survive a different sample.

REFERENCE | CLUSTERING | CONTRAST ONLY | PAGE 43

WHY THIS PAGE EXISTS

The models folder contains no clustering algorithm. This page is here so that **regression**, **classification** and **clustering** can be distinguished properly — not to imply that K-Means or DBSCAN were covered.

1 INTERNAL METRICS - NO LABELS REQUIRED

geometry, not truth

METRIC	WHAT IT MEASURES	READ IT AS…
Inertia	$\Sigma_i \lVert x_i - \mu_{c(i)} \rVert^2$	*Within-cluster spread. Lower is tighter, but it always falls as k grows — use the elbow, never the raw value.*
Silhouette	$(b - a) / \max(a, b)$	***a** = mean distance to own cluster; **b** = mean distance to the nearest other cluster. Range −1 to 1; above 0.5 is a clear structure.*
Davies–Bouldin	mean over clusters of $(s_i+s_j)/d_{ij}$	*Within-cluster spread relative to between-cluster distance. Lower is better; 0 is ideal.*
Calinski–Harash	$(SS_{between}/(k-1)) / (SS_{within}/(n-k))$	*Between-cluster separation divided by within-cluster spread. Higher is better; fast to compute.*

All internal metrics reward some shapes more than others: silhouette favours convex, well-separated blobs and will happily reject a correct density-based clustering of crescents.

2 EXTERNAL METRICS - WHEN LABELS EXIST

labels are a luxury

ADJUSTED RAND INDEX (ARI)

Agreement between the discovered clusters and known labels, corrected for chance. 1 = perfect, 0 = random. The default when you have a labelled benchmark.

NORMALISED MUTUAL INFORMATION (NMI)

How much knowing the cluster tells you about the label. 1 = perfect; not adjusted for chance, so compare with care.

DO NOT USE ACCURACY

Cluster ids are arbitrary: cluster 3 is not "wrong" because the truth called it 1. Accuracy requires a matching step and is still a poor summary — use ARI or NMI.

STABILITY IS THE HONEST TEST

Cluster a bootstrap sample and measure how often pairs of rows stay together (the Jaccard similarity of clusters). A solution that dissolves under resampling is not a finding.

3 THE WORKFLOW FOR CHOOSING K

numbers plus judgement

1. Scale features — clustering is distance-based, exactly like KNN.
2. Fit for a range of k and record inertia, silhouette and Davies-Bouldin.
3. Look for the elbow in inertia *and* a peak in silhouette; when they disagree, prefer silhouette for interpretability.
4. Check stability across bootstrap samples and random seeds.
5. Finally, inspect the cluster profiles: a statistically tidy cluster that makes no business sense is still useless.

PART 4 · THE REFERENCE SHELF · R6

The Master Table

All seven models on one page. Read down a column to compare a property; read across a row to remember a model.

COMPARISON · SEVEN MODELS · PAGE 44

1 SIDE BY SIDE

the whole guide, condensed

MODEL	TASK	DECISION SHAPE	SCALING	KEY KNOBS	EXPLAIN	EXTRAPOLATES	WATCH OUT FOR
Linear Regression	Regression	Straight line / flat plane	For penalties	features, α	High	Yes, cautiously	Non-linearity, outliers, omitted variables
Logistic Regression	Classification	Linear boundary in x	For penalties	C, threshold	High	Probability only	Separation, imbalance, the 0.5 reflex
Decision Tree	Both	Axis-aligned rectangles	No	depth, leaf size, ccp_alpha	High if shallow	No	Instability, overfitting, staircase boundaries
Random Forest	Both	Average of many partitions	No	trees, max_features	Medium	No	Size, uncalibrated votes, misleading importance
Extra Trees	Both	Average of random partitions	No	trees, max_features, leaf	Medium	No	Higher bias, random cuts missing a key threshold
KNN	Both	Local neighbourhoods	Mandatory	k, metric, weights	Medium (examples)	No	Slow prediction, curse of dimensionality
SVM	Both	Max-margin line or kernel curve	Mandatory	kernel, C, γ, ε	Low	No	Kernel choice, tuning cost, no native probability

2 AT A GLANCE

the shortcuts

MOST INTERPRETABLE	Linear · Logistic · shallow Tree	FASTEST TO TRAIN	Linear · Logistic · KNN (no training)	NEEDS SCALING	KNN · SVM · penalised linear
MOST ACCURATE OFF THE SHELF	Random Forest · Extra Trees	FASTEST TO PREDICT	Linear · Logistic · shallow Tree	HANDLES $p \gg N$	SVM (linear) · regularised linear

3 WHAT THE TABLE CANNOT TELL YOU

read it as a prior

Every entry is a *typical* behaviour, not a law. A random forest can extrapolate a little if the target is flat beyond the data; a linear model can fit a curve if you add polynomial features; KNN can be fast if n is small. The table is a starting point for an experiment, not a substitute for one.

THE ONLY FAIR COMPARISON

Same rows, same folds, same preprocessing budget, same tuning effort — and a test set opened once. Anything else measures your patience, not the model.

PART 4 · THE REFERENCE SHELF · R7

Which Model Should I Pick?

Start from the target and the data, not from the algorithm you read about last.

DECISION MAP · ONE PAGE · PAGE 45

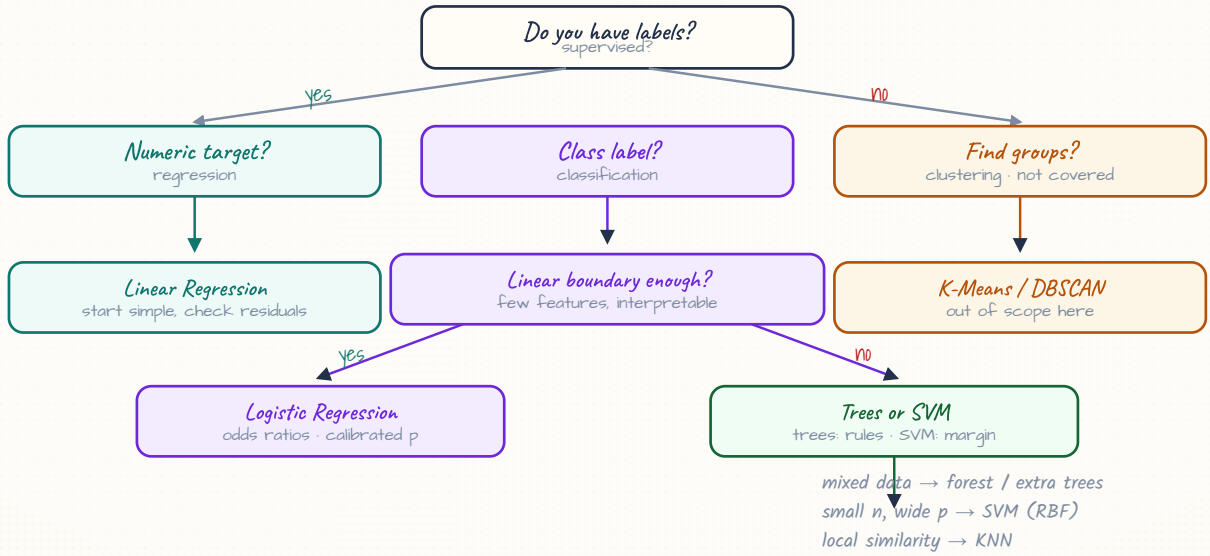


Fig. 43.1 — A first-pass decision map. It narrows the shortlist; it does not replace cross-validation.

1 FIVE QUESTIONS, IN ORDER

target first, algorithm second

1. **Do you have labels?** No → clustering or dimensionality reduction (not covered here). Yes → continue.
2. **Number or class?** Number → regression metrics; class → confusion-matrix metrics.
3. **Is the boundary plausibly linear?** Try the linear baseline first — it is fast and explains itself.
4. **How much data, how many features?** Small n with wide p favours SVM or regularised linear; large tabular data favours forests.
5. **What does a mistake cost?** That decides the metric, the threshold, and whether calibration is required.

- [ ] A baseline is defined and beaten.
- [ ] The metric matches the cost of error.
- [ ] The split respects groups and time.
- [ ] Scaling is inside the pipeline.
- [ ] The test set has not been touched.
- [ ] The winner is compared on identical folds.

2 DEFAULTS THAT USUALLY WORK

shortcuts, not rules

SITUATION	FIRST MODEL TO TRY
First look at a regression problem	*Linear regression + residual plots, then Ridge if features are many/correlated*
First look at a classification problem	*Logistic regression, with precision/recall at a chosen threshold*
Mixed tabular data, non-linear, medium n	*Random Forest — then Extra Trees if speed matters*
Small n, many features (text, omics)	*Linear SVM, or logistic regression with L2; RBF SVM if dense and scaled*
Similarity is the whole story	*KNN on scaled features or embeddings; check the k-curve*
You must explain one decision	*Shallow decision tree or logistic regression — not a forest path*

PART 4 · THE REFERENCE SHELF · R8

Hyper-parameters & Tuning

Tune the dial that changes the model's shape, in the right order, on the same folds — and stop when the gain is smaller than the noise.

PRACTICAL · ALL MODELS · PAGE 46

1 WHAT TO TURN, PER MODEL

the order matters

MODEL	TUNE IN THIS ORDER	LEAVE ALONE UNLESS…
Linear	*Feature engineering → α (if regularised)*	*fit_intercept, positive constraints*
Logistic	*Features → C → class_weight → threshold*	*solver (only if it fails to converge)*
Decision Tree	*max_depth → min_samples_leaf → ccp_alpha*	*criterion (Gini vs entropy rarely matters)*
Random Forest	*n_estimators (enough) → max_features → min_samples_leaf*	*max_depth (bagging already controls variance)*
Extra Trees	*n_estimators → min_samples_leaf → max_features*	*bootstrap (only for OOB)*
KNN	*Scaling → k → distance metric → weights*	*algorithm (auto-selects the tree structure)*
SVM	*Scaling → kernel → C and γ jointly (SVR: ε)*	*probability (only if calibrated risk is needed)*

2 SEARCH STRATEGY

pick one, document it

GRID SEARCH — Exhaustive, reproducible, expensive. Fine for 2–3 parameters with small grids.

RANDOM SEARCH — Samples distributions; usually finds a good region faster than a grid.

BAYESIAN / TPE — Uses past trials to choose the next; strong when a fit is expensive.

SUCCESSIVE HALVING — Kills weak candidates early; efficient for many configurations.

NESTED CV — Inner loop tunes, outer loop estimates generalisation — the honest score.

REPEATED CV — Different random folds; exposes a score that was a fold artefact.

3 RULES THAT PREVENT SELF-DECEPTION

tuning is a discipline

- **Tune on validation folds only.** The test set is opened once, after all choices are made.
- **Search on a log scale** for α, C, γ and learning-rate-like parameters.
- **Keep preprocessing inside the pipeline** so scaling and imputation are fitted per fold.
- **Stop when the gain is below the fold noise.** A 0.2-point improvement with a 1.5-point spread is not a result.
- **Record the search space**, the seed and the winning configuration — otherwise the score is not reproducible.
- **Report the tuned score and the default score.** If tuning bought nothing, say so.

PART 4 · THE REFERENCE SHELF · R9

Common Mistakes & Audit

Most bad models are not bad algorithms. They are good algorithms fed a leak, judged by the wrong number, or deployed without a baseline.

CHECKLIST · BEFORE YOU SHIP · PAGE 47

1 TWELVE TRAPS, IN THE ORDER THEY BITE

read this before every report

1. **Leakage before the split** — scaling, imputing or selecting features on all rows.
2. **Random splits on time data** — the model learns from the future.
3. **Duplicate rows across folds** — the same patient, session or image on both sides.
4. **Tuning on the test set** — even once, even "just to check".
5. **Accuracy on imbalanced data** — 98% by predicting the majority class.
6. **No baseline** — a score with nothing to compare against.
7. **Ignoring the fold spread** — treating noise as an improvement.
8. **Mistaking importance for causation** — especially with correlated features.
9. **Unscaled distance models** — KNN and SVM silently using only the largest unit.
10. **Uncalibrated probabilities** — a vote fraction treated as a risk.
11. **Explaining a forest by one tree** — a path that is not the model.
12. **Shipping without monitoring** — drift quietly turns a good model into a bad one.

2 FOUR AUDITS

four gates, sixteen checks

DATA AUDIT

- [ ] Split respects groups, time and duplicates.
- [ ] Target is not present in any feature, directly or as a proxy.
- [ ] Class balance and missingness checked per split.
- [ ] Feature ranges at training match production.

MODEL AUDIT

- [ ] Beats a dumb baseline by more than the fold noise.
- [ ] Hyper-parameters tuned on validation folds only.
- [ ] Scaling and imputation inside the pipeline.
- [ ] Learning curve inspected for data-hunger.

EVALUATION AUDIT

- [ ] Metric matches the cost of error.
- [ ] Threshold chosen, not defaulted.
- [ ] Errors reviewed by segment, not just aggregate.
- [ ] Probabilities calibrated if used as risk.

DELIVERY AUDIT

- [ ] Test set opened exactly once.
- [ ] Seed, search space and version recorded.
- [ ] Inference latency and memory measured.
- [ ] Monitoring and a retraining trigger defined.

3 RED FLAGS IN SOMEONE ELSE'S MODEL REPORT

how to read a claim

- **A single number, no baseline.** "92% accuracy" says nothing without the majority-class rate beside it.
- **Perfect or near-perfect scores.** On real tabular data, 0.99 usually means leakage, not genius.
- **One split, one seed.** No fold spread means no idea whether the score is stable.
- **Precision without recall** (or the reverse). One half of a trade-off is a sales pitch, not an evaluation.
- **AUC quoted for a deployment that uses a 0.5 threshold.** Ranking quality does not describe the shipped labels.
- **Importance without correlation checks.** Correlated features split their importance and look harmless.

THE ONE QUESTION THAT EXPOSES MOST OF THEM

"What exactly was in the training set, and when was the test set last touched?" If the answer is vague, the score is not evidence yet.

PART 4 · THE REFERENCE SHELF · R10

Glossary

The vocabulary of the guide, in plain English — forty-eight terms worth being able to define without notes.

REFERENCE VOCABULARY PAGE 48

Bagging — Training many models on bootstrap samples and averaging them to cut variance.

Baseline — The score of a trivial rule — mean, majority class — that any real model must beat.

Bias — Error from a model too simple to represent the truth.

Bootstrap — Sampling n rows with replacement; about 63% of distinct rows appear.

Calibration — Whether predicted probabilities match observed frequencies.

CART — Classification and Regression Trees — the greedy binary-split algorithm.

Class imbalance — One label is far rarer than another, so accuracy flatters a lazy model.

Class weight — A multiplier on the loss that makes rare classes count more.

Cross-validation — Rotating train/validation folds to estimate generalisation.

Curse of dimensionality — In high dimensions, distances stop being informative and KNN degrades.

Decision boundary — The surface where a classifier switches from one class to another.

Discriminative — Modelling p(y|x) directly rather than the data distribution.

Elastic Net — A penalty mixing L1 and L2 regularisation.

Entropy — Impurity measure: $-\sum p \log p$; 0 for a pure node.

Feature — An input variable (column) used by the model.

Gini — Impurity measure $1 - \sum p^2$; cheap and standard for trees.

Hinge loss — $\max(0, 1 - y \cdot f(x))$; the SVM's per-row penalty.

Hyper-parameter — A setting chosen before fitting, e.g. k or C.

Imputation — Filling missing values, ideally inside the CV pipeline.

Instance-based — Storing training rows and predicting from similar ones (KNN).

Intercept — The prediction when every feature is zero; the b in $y = wx + b$.

Kernel — A similarity function equal to an inner product in a richer space.

Leakage — Information from the test set or the future contaminating training.

Log loss — Binary cross-entropy; punishes confident wrong probabilities.

Margin — Empty band between the classes and the SVM boundary.

MSE — Mean squared error; the loss OLS minimises, in squared units.

Multicollinearity — Features that are strongly correlated, so coefficients become unstable.

Non-parametric — A model whose complexity grows with the data, not a fixed formula.

Odds ratio — e^{β}: multiplicative change in odds per unit of a feature.

One-hot encoding — Turning a categorical column into one 0/1 column per level.

Overfitting — Low training error, high validation error — high variance.

Outlier — A row far from the rest; it can dominate a loss or a distance.

Pipeline — A chain of preprocessing and model steps fitted together per fold.

Pruning — Removing branches that do not pay for their complexity.

R^2 — Share of variance explained; can be negative on unseen data.

Regularisation — A penalty on model complexity, usually on coefficient size.

Residual — $y - \hat{y}$; what the model missed for one row.

Scaling — Putting features on comparable ranges — mandatory for KNN and SVM.

Seed — The number that makes a random split or forest reproducible.

Soft margin — An SVM that tolerates some violations, priced by C.

Standardisation — Subtract the mean, divide by the standard deviation.

Support vector — A training row on or inside the margin; it defines the SVM.

Threshold — The score cut-off that turns a probability into a class label.

Underfitting — High training and validation error — the model is too simple.

Validation set — Rows used to compare models and tune settings, never to fit coefficients.

Variance — Error from sensitivity to the particular training sample.

Voting (hard/soft) — Combining classifiers by majority label or by averaged probability.

If two terms are still confusable, write the one-sentence distinction on a card. The five that repay the effort most:

BIAS VS VARIANCE	Bias is too simple; variance is too twitchy. Validation error tells you which.
PRECISION VS RECALL	Precision: when you fire, are you right? Recall: did you fire at all?
BAGGING VS BOOSTING	Bagging averages independent models; boosting fits each model to the last one's errors.
PARAMETER VS HYPER-PARAMETER	A parameter is learned (weights); a hyper-parameter is chosen (k, C, depth).
PROBABILITY VS CLASS	A probability ranks rows; a threshold decides. They are different products.
FEATURE VS TARGET	The feature is what you know at prediction time; the target is what you are trying to know.

PART 4 · THE REFERENCE SHELF · R11

One-Page Revision

Everything that matters, compressed to a single sheet. If you can rebuild this page from memory, you are ready.

REVISION · FINAL PAGE · PAGE 49

1 THE SEVEN MODELS IN SEVEN LINES

say each one out loud

Linear Regression — weighted sum, least squares; straight boundary; coefficients as effects; check residual shape.

Logistic Regression — linear log-odds, sigmoid, log loss; probability plus threshold; precision/recall at the operating point.

Decision Tree — recursive axis-aligned splits by impurity; readable rules; unstable and non-extrapolating.

Random Forest — bagged deep trees with random feature subsets; strong tabular default; OOB, importance, no extrapolation.

Extra Trees — random split thresholds, no bootstrap; faster and more decorrelated, slightly higher bias.

KNN — vote or average of k nearest rows; the distance metric is the model; scaling mandatory, high dimensions fatal.

SVM — maximum-margin boundary; support vectors define it; kernels for curves; scale first, tune C and γ.

2 THE FORMULAS WORTH REMEMBERING

write them, don't read them

$\hat{y} = X\beta$ · *OLS minimises* $\|y - X\beta\|^2$

$p = \sigma(w \cdot x + b) = 1/(1+e^{-z})$

$Gini = 1 - \Sigma p_k^2$ · $IG = I_p - \Sigma(n_c/n) I_c$

$margin = 2/\|w\|$ · $hinge = \max(0, 1 - y\, f(x))$

$RMSE = \sqrt{(SSE/n)}$ · $R^2 = 1 - SSE/SST$

$P = TP/(TP+FP)$ · $R = TP/(TP+FN)$ · $F1 = 2PR/(P+R)$

$E[(y-\hat{y})^2] = bias^2 + variance + \sigma^2$

$Var_{forest} = \rho\sigma^2 + (1-\rho)\sigma^2/B$

3 THE LAST FIVE CHECKS

then ship

- [ ] Baseline beaten by more than the fold noise.
- [ ] Metric matches the cost of a false positive/negative.
- [ ] Scaling and imputation inside the pipeline.
- [ ] Test set opened once, at the end.
- [ ] Predictions explained with the actual neighbours, path, or coefficients — not a story.

NOTES COMPILED BY

Chandan Chaudhari

github.com/chandanc5525 · November 2026

Seven models, one template, and a healthy distrust of a single number. The next step is not another algorithm — it is a dataset you care about, a baseline, and an honest split.